

1 Module Configuration (INTERNAL REVIEW ONLY)

Resolved for module instance: MRAMC

Resolved using instance file: mramc_spec/spec_source/inst/mramc.inst

Parameter	Instantiated Value	Range	Origin	Description
RTL PARAMETERS & SIGNALS				
DID_W	0x4 (4)	[1-8]	RTLLOC	RDC Domain ID width
LARGE_PART_SIZE	0x15 (21)	[8-21 (0x15)]	RTLLOC	Size of mram large partition in log2, ex: value 20 means actual partition size is $2^{20} = 2 \text{ MB}$
MAX_NE_W	0x4 (4)	[0-4]	RTLLOC	Maximum size of field NUM_ENTRIES in the register Pi_RIA_Wj
MAX_RIA_NUM	0x4 (4)	[0-4]	RTLLOC	Maximum number of the registers Pi_RIA_Wj by AXI Port
MRAM_DUAL_PORT	true	T/F	RTLLOC	Indicates MRAM has dual read ports when set to 1
NE_W_0	0x1 (1)	[1-4]	RTLLOC	Size of field NUM_ENTRIES in the register P0_RIA_Wj
NE_W_1	0x4 (4)	[1-4]	RTLLOC	Size of field NUM_ENTRIES in the register P1_RIA_Wj
NE_W_2	0x4 (4)	[1-4]	RTLLOC	Size of field NUM_ENTRIES in the register P2_RIA_Wj
NE_W_3	0x3 (3)	[1-4]	RTLLOC	Size of field NUM_ENTRIES in the register P3_RIA_Wj
NUM_AXI_PORTS	0x4 (4)	[1-4]	RTLLOC	Number of AXI target ports
NUM_LARGE_PART	0x10 (16)	[1-32 (0x20)]	RTLLOC	Number of mram large partitions
NUM_SMALL_PART	0x4 (4)	[1-4]	RTLLOC	Number of mram small partitions
RIA_NUM_0	0x0 (0)	[0-4]	RTLLOC	Number of the registers P0_RIA_Wj
RIA_NUM_1	0x2 (2)	[0-4]	RTLLOC	Number of the registers P1_RIA_Wj
RIA_NUM_2	0x2 (2)	[0-4]	RTLLOC	Number of the registers P2_RIA_Wj

Table continues on the next page...

Table continued from the previous page...

Parameter	Instantiated Value	Range	Origin	Description
RIA_NUM_3	0x1 (1)	[0-4]	RTLLOC	Number of the registers P3_RIA_Wj
SMALL_PART_SIZE	0x12 (18)	[18 (0x12)-18 (0x12)]	RTLLOC	Size of mram small partition in log2, ex: value 18 means actual partition size is $2^{18} = 256$ KB
SPEC PARAMETERS				
Glossary	false	T/F	DIS	When true, includes the glossary in the chapter.
SHOW_SECURE	false	T/F	DIS	Shows security features. This should be set to false so security information is not shown to customer.
spec_tag	false	T/F	DIS	Set to true to show the specification tagging in documentation.
DERIVED PARAMETERS				
NUM_LARGE_PART_LOCK	0x8 (8)	[0-unbounded]	N/A	Number of write lock bits for large partitions. For dual MRAM read port config, this is lock bits in lower or upper partitions and not combined. For example, when there are 5 lock bits for lower and upper partitions each, this parameter value will be 5.
AGEN IMPLICIT PARAMETERS				
hierNickname	MRAMC		AGEN	DEPRECATED. Use getInstHierNickname(). A hierarchical nickname of the instance. It is a dot-separated string of nicknames of all parent instances in instance list. E.g. if there is <instance name="A" nickname="a"> <instance name="B"> <reg_instance name="C" nickname="c">, hierarchical nickname of <reg_instance> is "a.c". Empty for instances not loaded from the instance list.
module	spec_source		AGEN	DEPRECATED. Use getInstModule(). A module name of the instance. Equals to attribute 'module' in instance list for instances loaded from instance list. Empty, if not specified or an instance not loaded from instance list.
moduleNickname			AGEN	DEPRECATED. Use getInstModuleNickname(). A module nickname of the instance. Equals to attribute 'module_nickname' in instance list for instances loaded from instance list or module name derived from metadata. For other instances, it is empty.
name	MRAMC		AGEN	DEPRECATED. Use getInstName(). A name of the instance. Equals to attribute 'name' in instance list for instances loaded

Table continues on the next page...

Table continued from the previous page...

Parameter	Instantiated Value	Range	Origin	Description
				from instance list. For other instances, it is the source file name without path and extension.
nickname	MRAMC		AGEN	DEPRECATED. Use getInstNickname(). A nickname of the instance. Equals to attribute 'nickname' in instance list for instances loaded from instance list. For other instances, it is empty.
GLOBAL PARAMETERS				
integration_info	true	T/F	DIS	Boolean parameter used to show/hide notes specifically targeted at the SOC/Subsystem integration lead
internal	true	T/F	DIS	Boolean parameter to use when no specific internal target audience is identifiable.
ip_design_info	true	T/F	DIS	Boolean parameter used to show/hide notes specifically written by the IP designer to help explain internal design information, design decisions, etc.
security_sensitive_info	false	T/F	DIS	Boolean parameter to show/hide security sensitive information.
show_paramdef_tables	true	T/F	DIS	Boolean parameter for documentation control - used to show/hide paramdef visibility tables contained in IP block guides; These tables can be independently turned on to allow documents to be built as "customer facing" documents but with these tables enabled for debugging parameter settings.
show_revision_history	true	T/F	DIS	Boolean parameter for documentation control - independent switch to enable/disable revision history information
test_function_info	true	T/F	DIS	Boolean parameter used to show/hide notes, registers, or other test functionality specifically targeted at PE/TE or verification;
third_party_enablement_info	true	T/F	DIS	Boolean parameter used to show/hide notes specifically targeted at third party enablement tools vendors (debuggers, compilers, etc.)

Legend:

- RTLPAR - RTL_PARAMETER
- RTLSIG - RTL_SIGNAL
- RTLDEF - RTL_DEFINE
- RTLLOC - RTL_LOCALPARAM
- DCFG - DESIGN_CONFIG_OPTION

- BCFG - BUSINESS_CONFIG_OPTION
- DIS - OUTPUT_DISPLAY_CONTROLS
- AGEN - AGEN_IMPLICIT_PARAMETER
- N/A - NOT_APPLICABLE (Derived parameter)
- NS - NOT_SPECIFIED (Please set parameter origin to an appropriate value)
- T/F - TRUE/FALSE (Boolean parameter values)

2 Overview

MRAMC supports 64-bit and 128-bit data interface with system bus and 256-bit data interface with Magnetic Random Access Memory (MRAM). It converts system bus transactions to MRAM transactions. MRAMC supports read and write of MRAM with triple, dual, single pipe reads and octo page, quad page, full page, partial page writes (byte, word and double word writes).

2.1 Block diagrams

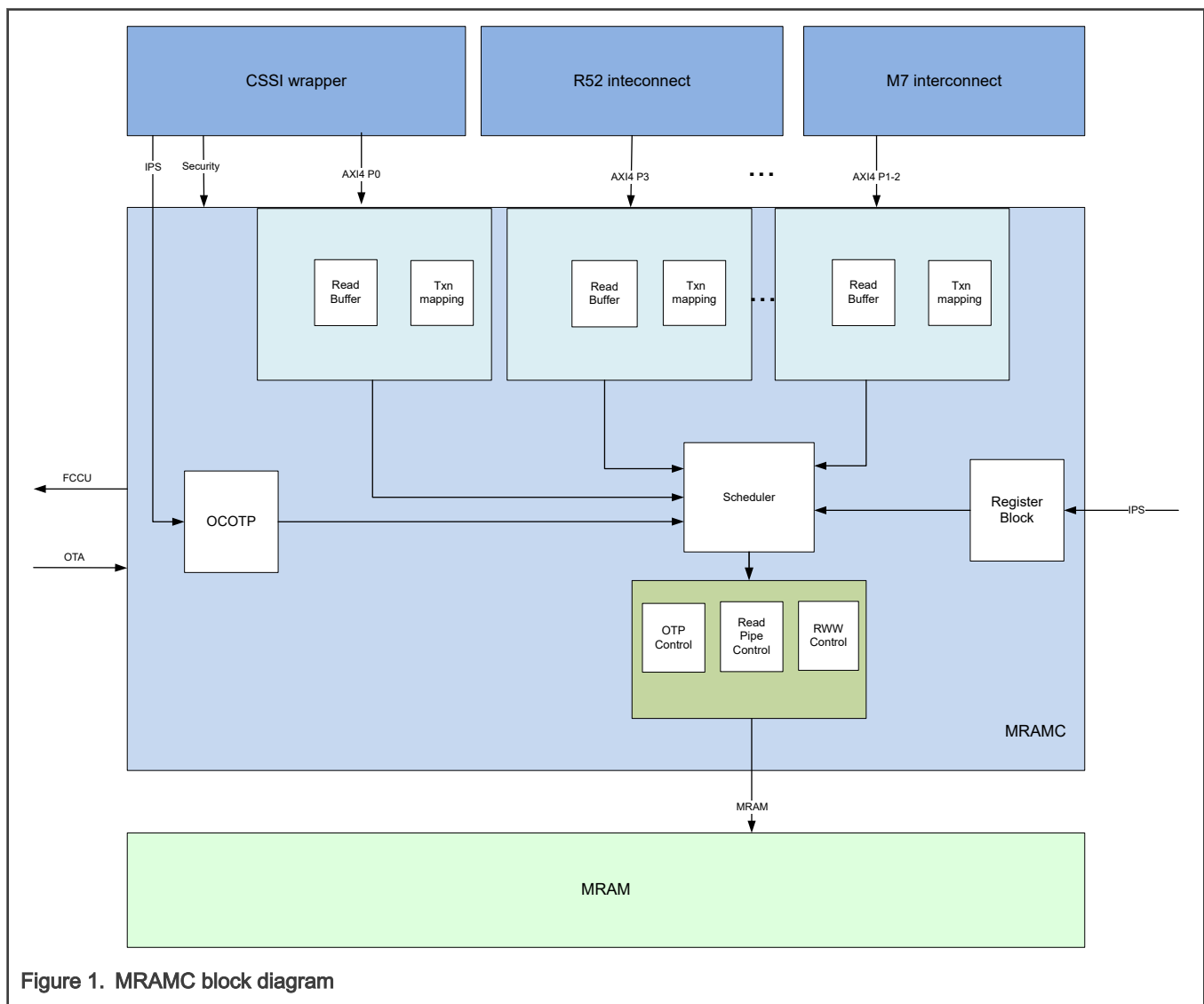


Figure 1. MRAMC block diagram

2.2 Features

- Support 33 MB of MRAM memory
- 64-bit system bus interface with application sub-system
- 128-bit system bus interface with compute sub-system
- 64-bit system bus interface with security sub-system
- MRAM interface supports dual 256-bit read and single write interface bus
- Access to MRAM large and small partition space (referred to as MRAM space from here on), OTP space and UTEST space.
- MRAM partition lock protection
- Configurable read line buffering and prefetching support via a mini-cache, for each system port to provide quick buffer hit read response
- Single entry 32 bytes wide gathering write buffer on system bus interface ports
- Over the Air (OTA) for active and passive image swap

3 Functional description

MRAMC has interfaces between:

- The system bus port
- The magnetic memory array

MRAMC is used to connect MRAM to the system. It acts as a bridge that converts system transactions to MRAM interface transactions. MRAMC supports triple, dual, single pipe reads and octo page, quad page, full page, partial page writes to MRAM.

For system read transaction, MRAMC captures read data from the magnetic memory array and drives it onto the system bus. MRAMC has buffer to store pages of data from magnetic memory. MRAMC buffer supports prefetching of page from memory, page may be prefetched in advance of being requested, allowing quick read data responses on buffer hits.

3.1 Memory organization

MRAM address space is composed of 16 large partitions and 4 small partitions. Each large and small partition contains normal MRAM space, UTEST space, and OTP space.

All large and small partitions are organized between two banks namely upper and lower banks. Large partitions from upper and lower bank are grouped to a partition pair. Small partitions are not grouped in a pair.

[Table 1](#) shows example for 16 large partitions, but it can be interpreted accordingly for a specific configuration.

Table 1. Example for 16 large partitions

Large partition pair number	Large partition number for upper bank	Large partition number for lower bank
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7

Table 2 shows example for 4 large partitions, but it can be interpreted accordingly for a specific configuration.

Table 2. Example for 4 large partitions

Small partition number for upper bank	Small partition number for lower bank
0	0
1	1

MRAMC maps these MRAM partitions in logical memory map. Large partition normal MRAM space is interleaved between partitions in a pair on 32 bytes granularity. All other space, large partition UTEST and OTP space, and small partition normal MRAM space and UTEST and OTP space is not interleaved, meaning pages are arranged consecutively.

3.1.1 Interleaving

Table 3 shows interleaving scheme for partition pair #0 (upper and lower partitions #0 each), and similarly applies to other pairs. Logical consecutive 8 pages (each page is 32 bytes) are interleaved by MRAMC between lower and upper partition pair. This interleaving is done by MRAMC hardware and is transparent to your software, and the space is represented in contiguous format such that user do not have to deal with any data manipulation.

Table 3. interleaving scheme for partition pair #0

Large partition #0 for upper bank	Large partition #0 for lower bank
Address 20h-3Fh	Address 00h-1Fh
Address 60h-7Fh	Address 40h-5Fh

3.1.2 Non-interleaving

Non-interleaving scheme is used for access to all space except large partition normal MRAM space. Table 4 shows example of logical consecutive 4 pages (each page is 32 bytes) organized consecutively in partition #0 in lower bank, and similar can be interpreted for upper bank.

Table 4. Example of logical consecutive 4 pages

Partition #0 for lower bank
Address 00h-1Fh
Address 20h-3Fh
Address 40h-5Fh
Address 60h-7Fh

MRAMC memory map base address is 0x0800_0000.

Table 5. MRAMC memory map

Byte address	Contents	Size	Additional information
0h-003F_FFFFh	MRAM large partition pair 0	4 MB	Code partitions are interleaved on 32B between lower and upper. Pair #0 here refers to pair of upper and lower partition #0, similarly for other large partitions. Pair has 2 partitions of 2 MB each.
0040_0000h-01FF_FFFFh	MRAM large partition pairs 1 to 7	28 MB	

Table continues on the next page...

Table 5. MRAMC memory map (continued)

Byte address	Contents	Size	Additional information
0440_4000h-0440_7FFFh	OTP space from lower large partitions 4 to 7	16 KB	
0440_8000h-0440_FFFFh	OTP space from upper large partitions 0 to 7	32 KB	
0480_0000h-0483_FFFFh	MRAM lower small partition 0	256 KB	
048F_E000h-0490_7FFFh	UTEST space from small and large partitions	40 KB	2 KB from each partition in this ascending address order: • Lower small partitions 0 to 1 • Upper small partitions 0 to 1 • Lower large partitions 0 to 7 • Upper large partitions 0 to 7

3.2 Read transactions

On an incoming system read request, a read mini-cache (read buffer) lookup is performed. If a buffer hit occurs, the requested data is retrieved from the previously loaded buffer entry and returned on the system bus. If a buffer miss occurs, MRAM memory access is initiated.

3.3 Write transactions

Writes to MRAM can be initiated through system bus or register bus interface. Register bus interface is the recommended way of writing to MRAM.

3.3.1 System bus MRAM write

System bus can be used to initiate write transactions to MRAM.

There is a single entry page buffer for write per system port. This write buffer merges writes to the same page address. Write buffer entry is written to MRAM (flushed or evicted) on one of these conditions for that specific port:

- New system bus write does not hit to page address in write buffer.
- Write buffer entry has all page data bytes valid. If at end of transactions, buffer has partial page data then it won't be auto flushed, so you need to use other flush mechanisms.
- Software flush initiated by writing to PCn[IB].
- System bus read hits in write buffer. Read is kept on hold until write buffer is flushed.

In order to guarantee the write buffer content is eventually written to MRAM, you should use one of the above ways to flush it.

NOTE

Writing less than page size reduces write endurance of MRAM. So it is strongly recommended to restrict system bus writes to minimum one page size.

NOTE

You must not use system bus for writing to OTP space, but use IPS write path instead.

On an incoming system write request, a read mini-cache lookup is performed. If a read buffer hit occurs, the read buffer entry is invalidated.

3.3.2 Register bus MRAM write

Register bus write to MRAM supports single, quad or octo pages write.

Table 6 describes restriction on number of pages that can be written to specific memory space. See SoC memory map for addressing of these spaces.

Table 6. MRAMC assignment policy

MRAM Memory space	Valid value of number of write pages
Code	1 or 8
Data	1 or 4 or 8
OTP	1
UTEST	1

There are two set of write buffer registers for initiating write to MRAM. Each set consists of these registers:

- Write address
- Write data
- Write strobe
- Write control
- Write status

You can use any of the set for write to MRAM. Two sets can be used such that while one set is busy with the ongoing write, the other set can be configured.

3.3.2.1 Write sequence lock

There is write sequence lock for each set of write buffer registers. The sequence is locked or owned by the domain ID of initiator that first gets semaphore by successfully writing to Write address register. Initiator can check WSTATn[WID] to check against its own domain ID to determine if it has gained the semaphore to write sequence. This semaphore mechanism is useful for supporting multiple initiators trying to write MRAM at the same time.

Write sequence lock is released in one of the following ways:

- Write operation is finished when WSTATn[WBUSY] is set to 0 by hardware.
- Write sequence lock is released by owning domain ID by writing 1 to WCTLn[CWSL], before WCTLn[SMW] is set to 1.
- Write sequence watchdog times out.

3.3.2.2 Write sequence watchdog timer

There is watchdog timer for each set of write buffer registers. Watchdog timer is helpful to prevent denial of service in case a thread has owned the write sequence but fails to finish the write sequence.

Watchdog timer resets and starts counting once write sequence lock is acquired. It stops when WCTLn[SMW] or WCTLn[CWSL] is written 1, or it times out. Write sequence lock is released if watchdog timeout occurs.

3.3.2.3 Write address access evaluation

Write address space is accessible by multiple masters by using the write buffer registers. XRDC MRC is used along with write buffer for isolation of address space between initiators. This MRC needs to be configured by your software for isolation.

Write to WADDRn is routed through MRC for access evaluation, and write finishes without error if it has access but is terminated with error if it does not have access. Write sequence lock is not acquired if the write results in an error.

3.3.2.4 Register bus write programming sequence

Register bus write programming sequence is as described below

1. Write to WADDRn to hold address of the first page to be written and to gain semaphore for the write sequence.
2. Read WSTATn[WDID] to check if it contains the same domain ID as initiator. Same ID indicates the current initiator is owning the write sequence. If not same, go to previous step.
3. Read WSTATn to check for error set in WDTE and WERR, from previous write sequence. If error is set then wait for some time to allow previous sequence owner to clear them. If error is still set then write 1 to clear them.
4. Write 1 to WCTLn[WCLR] if you want to reset WDATAAn_PAGEn_WORDn and WSTRBn_PAGEn to default value.
5. Write to WDATAAn_PAGEn_WORDn for storing content of up to eight pages to be written to MRAM.
6. Write to WSTRBn_PAGEn if value different than current is needed.
7. Write to WCTLn with value of number of write pages to be written in NUM_WP.
8. Write 1 to WCTLn[SMW] to initiate write. This step can be combined with previous step.
9. Wait for WSTATn[WBUSY] to be cleared which indicates completion of write.
10. Read WSTATn[WERR] to check whether command completed successfully or terminated with an error.

3.4 Write partition lock protection

MRAMC provides a way to protect MRAM partitions from being accidentally modified. There are lock registers, for various address spaces like small and large MRAM partitions and OTP, that prevents write if lock bit set to value 1. These lock bits are shared by multiple initiators, and there are protect lock registers to gain ownership of lock bit.

MRAMC does not block or generate error for access to locked partition but it forwards transaction to MRAM where it is handled. See MRAM chapter for details.

Below table describes state change to protect lock and lock register fields.

Table 7. Lock state change

Current PLCK state	PLCK state change allowed	LCK change allowed	Initiator allowed
00	01 or 10 or 11	0 to 1, or 1 to 0	Any domain ID
01	00 or 10 or 11	0 to 1, or 1 to 0	Owning domain ID
10	11	0 to 1, or 1 to 0	Owning domain ID
11	Write ignored	0 to 1 allowed, 1 to 0 not allowed	Any domain ID

3.5 Line read buffers and prefetch operation

Each MRAMC system port contain a read mini-cache or buffer. MRAMC uses the buffer for both prefetch and normal demand read fetches. The buffers are shared for code and data fetches, and can be controlled independently for code and data from control registers.

Prefetch triggering is controllable on a per-port basis from control registers. MRAMC prefetchs next line (page) from MRAM upon current system read transaction address hits read buffer but next line is not available in read buffers when prefetching is enabled.

System port's mini cache entry width is sized to match size of the processor cache line size associated with the system port. Mini-cache is implemented as a fully associative cache, with least-recently-used replacement policy. When mini-cache is full and a read miss occurs then last recently used (LRU) entry is evicted and its content replaced by data read from MRAM.

Read buffer configuration

System bus port number	Buffer entry depth	Buffer entry width
1	16	32 bytes
2	16	32 bytes
3	8	64 bytes

3.5.1 Read buffer ID assignment

MRAMC supports assignment of read buffer entries to domain IDs individually per system bus port.

The assignment of buffer entries to domain IDs is software programmable with Port read ID assignment registers. If these registers are not enabled then all buffer entries for that port is used by all domain IDs.

Your software must assign buffer entries to domain IDs exclusively, meaning that listed domain IDs must use only assigned buffer entries and conversely unlisted domain IDs must use unassigned buffer entries.

MRAMC assignment policy for various options of assignment registers instantiated and applicable per port:

Table 8. MRAMC assignment policy

Number of registers	Register status	Assignment policy
Register 1	Enabled	Buffer entries listed in register 1 are used by domain IDs listed in register 1. All unlisted buffer entries are used by all unlisted domain IDs
Register 1	Disabled	All buffer entries per port are shared by all domain IDs.
Register 1 Register 2 (only applicable for multiple registers)	Both enabled	Buffer entries listed in register 1 are used by domain IDs listed in register 1. Buffer entries listed in register 2 are used by domain IDs listed in register 2. All unlisted buffer entries are used by all unlisted domain IDs.
Register 1 Register 2 (only applicable for multiple registers)	Enabled Disabled	Buffer entries listed in register 1 are used by domain IDs listed in register 1. All unlisted buffer entries are used by all unlisted domain IDs.
Register 1 Register 2 (only applicable for multiple registers)	Both disabled	All buffer entries are shared by all domain IDs.

A good example of using two assignment registers on port with 16 buffer entries being accessed by up to 16 domain IDs with resulted buffer entries to domain ID assignment policies:

Table 9. Example of using two assignment registers

Assignment registers	Content
Register 1	ENABLE=1, NUM_ENTRIES=5, MASK_ID=16'b0000000000001111
Register 2	ENABLE=1, NUM_ENTRIES=5, MASK_ID=16'b00000000011110000

Table 10. Assignment policies

Assignment policies	Description
Policy 1	Six buffer entries are explicitly assigned to domain IDs: 0-3
Policy 2	Six buffer entries are explicitly assigned to domain IDs: 4-7
Policy 3	Remaining four buffer entries are assigned to remaining domain IDs: 8-15.

3.6 Over the air (OTA)

MRAMC supports swap of code partitions and corresponding lock registers for OTA updates. Active and passive images can be stored different code spaces. Active image is always associated with lower logical address space, and MRAMC maps it to corresponding physical partition depending on OTA configuration.

If OTA feature is not enabled, then the entire code space is fixed.

If OTA feature is enabled, active and passive MRAM partitions are decided based on OTA_SEL input configuration to MRAMC. MRAMC supports full swap and partial swap.

For full swap, entire code space is split evenly between active and passive space. First half (lower) logical address is always in active region and second half logical address lies in passive region.

For partial swap, first 2 large partition pairs are fixed and the rest space is evenly split between active and passive space.

Passive image is updated while code executes from active image, and once the passive image is ready, device switches the OTA select and treat previously passive image as active. MRAMC remaps physical partition select to appropriate logical address space. When physical partition is swapped, MRAMC also swaps lock register control to MRAM, for example, when partition 0 is swapped with partition 2, lock registers are also swapped so that partition 0 lock registers always stay associated with active logical lower address space regardless of OTA configuration, and swap is transparent to user.

3.7 Clocking

MRAMC has no clocking considerations.

3.8 Error handling

Any error encountered for a transaction is captured in ERRSTATn, and the transaction error response is sent on the bus as well. Either alarm is raised to FCCU or interrupt is asserted, see ERRSTAT0 for details.

Your software need to clear ERRSTATn once it is finished with error exception handling.

3.9 Interrupts

MRAMC supports active high level interrupt. It is asserted when error is captured in specific ERRSTAT0 register fields. See the register section for specific fields that asserts interrupt.

3.10 Safety

There are various safety mechanisms implemented in MRAMC.

- The system bus interfacing with application sub-system is protected with odd parity.
- The system bus interfacing with compute sub-system is protected with DED (upto double bit error detection) ECC.
 - Request address and control coming to MRAMC are protected with DED ECC.
 - Read data provided by MRAMC is protected with DED ECC.
- Address and data traversing through MRAMC is protected with DED ECC.
 - Request address and control traversing through MRAMC are protected with DED ECC.
 - Read data traversing through MRAMC is protected with DED ECC.
- Interface between MRAMC and MRAM is protected with:
 - Write request address is protected with redundancy of address signals.
 - Write data is not protected by hardware. Software needs to read back data after write to check write data sanity.
 - Read address is protected with address feedback from MRAM that it used for actual read of MRAM array. MRAMC uses the feedback address to check against what is sent to MRAM.
 - Read data is protected by QED (upto Quad bit error detection) ECC.
- Any error detected through these safety mechanism triggers an alarm assertion to FCCU and error information is stored in ERRSTATn. Alarm signal is de-asserted when FCCU asserts clear signal.
- Error injection provides a way to test safety mechanisms and error reporting functionality. See Error Injection Control register for error injection.
- See MRAM chapter for safety mechanisms implemented within MRAM.

4 External signals

This module has no external signals.

5 Initialization

MRAMC does not need any initialization for it to operate. However, there are various configuration registers in MRAMC that you can configure.

6 Application information

6.1 Coherency

MRAMC contains read buffer, or write buffer, or both on each system bus port and register bus port. This means that data coherency between these buffers need to be handled.

You should not initiate read transaction while write transaction to the same address has not finished, and vice versa. Your software need to use software semaphore or flags or other mechanisms to take care of this. This is applicable regardless of read and write to same port or between multiple ports.

6.1.1 Coherency on same port

MRAMC handles data coherency between read and write transactions on the same port.

6.1.2 Coherency between multiple ports

MRAMC does not handle data coherency between multiple ports. Your software need to handle data coherency between multiple ports. Your software can do this by keeping track of address locations that are shared between the ports, and invalidating buffers in other ports before initiating transaction on a port.

For example, system bus port 1 and register bus port are sharing the address location. You can consider system bus port instead of register bus port for the example below.

- System port 1 initiated a transaction to an address location.
- Later, register port want to initiate transaction to the same address location. So, you should invalidate buffers in system port 1 before initiating transaction on register port. You can invalidate buffer on system port 1 by writing PC1[IB] to 1 and waiting for it to be cleared to 0 by hardware. You do not need to invalidate buffer in other ports if this address is not shared with those ports.
- Initiate transaction from register port.

Above coherency handling mechanism is just one example, and your software can use other mechanism that guarantees coherency.

7 MRAMC register descriptions

MRAMC provides an Internal Peripheral System (IPS) programming model that consists of control and status registers.

You can reference the programming model only by using a 32-bit (word) access. Transactions that are attempted using different access sizes, or to undefined (reserved) addresses generates an IPS error termination.

Write to read only registers don't generate error termination.

7.1 MRAMC memory map

MRAMC base address: 0h

Offset	Register	Width (In bits)	Access	Reset value
4h - Ch	Port Configuration (PC1 - PC3)	32	RW	0000_1003h
90h	Port i Read ID Assignment Word j (P1_RIA_W0)	32	RW	0000_0000h
94h	Port i Read ID Assignment Word j (P1_RIA_W1)	32	RW	0000_0000h
A0h	Port i Read ID Assignment Word j (P2_RIA_W0)	32	RW	0000_0000h
A4h	Port i Read ID Assignment Word j (P2_RIA_W1)	32	RW	0000_0000h
B0h	Port i Read ID Assignment Word j (P3_RIA_W0)	32	RW	0000_0000h
100h	Large Partition Lock (LPLOCK)	32	RW	0000_00FFh
110h	Small Partition Lock (SPLOCK)	32	RW	0000_0001h
118h	Large Partition OTP Lock (LPOLOCK)	32	RW	0000_0001h
11Ch	UTest Lock (UTLOCK)	32	RW	0000_0001h
130h	Protect Large Partition Lock Register (PLPLOCK0)	32	RW	0000_0000h
140h	Protect Small Partition Lock Register (PSPLOCK)	32	RW	0000_0000h
148h	Protect Large Partition OTP Lock Register (PLPOLOCK)	32	RW	0000_0000h

Table continues on the next page...

Table continued from the previous page...

Offset	Register	Width (In bits)	Access	Reset value
14Ch	Protect UTest Lock Register (PUTLOCK)	32	RW	0000_0000h
200h - 21Ch	Large Partition Lock Owner (LPLOCKOW0 - LPLOCKOW7)	32	R	0000_001Fh
300h	Small partition Lock Owner (SPLOCKOW)	32	R	0000_001Fh
304h	Large partition OTP Lock Owner (LPOLOCKOW)	32	R	0000_001Fh
308h	UTest Lock Owner (UTLOCKOW)	32	R	0000_001Fh
380h	Error Injection Control (ERRINJCTL)	32	RW	0000_0000h
388h	Error Status 0 (ERRSTAT0)	32	RW	0000_0000h
38Ch	Error Status 1 (ERRSTAT1)	32	R	0000_0000h
3A0h	Error Notification Control 0 (ERRNTCTL0)	32	RW	0001_01FCh
3FCh	Write Watchdog (WWDOG)	32	RW	0000_0000h
400h	Write Data (WDATA0_PAGE0_WORD0)	32	RW	FFFF_FFFFh
404h	Write Data (WDATA0_PAGE0_WORD1)	32	RW	FFFF_FFFFh
408h	Write Data (WDATA0_PAGE0_WORD2)	32	RW	FFFF_FFFFh
40Ch	Write Data (WDATA0_PAGE0_WORD3)	32	RW	FFFF_FFFFh
410h	Write Data (WDATA0_PAGE0_WORD4)	32	RW	FFFF_FFFFh
414h	Write Data (WDATA0_PAGE0_WORD5)	32	RW	FFFF_FFFFh
418h	Write Data (WDATA0_PAGE0_WORD6)	32	RW	FFFF_FFFFh
41Ch	Write Data (WDATA0_PAGE0_WORD7)	32	RW	FFFF_FFFFh
420h	Write Data (WDATA0_PAGE1_WORD0)	32	RW	FFFF_FFFFh
424h	Write Data (WDATA0_PAGE1_WORD1)	32	RW	FFFF_FFFFh
428h	Write Data (WDATA0_PAGE1_WORD2)	32	RW	FFFF_FFFFh
42Ch	Write Data (WDATA0_PAGE1_WORD3)	32	RW	FFFF_FFFFh
430h	Write Data (WDATA0_PAGE1_WORD4)	32	RW	FFFF_FFFFh
434h	Write Data (WDATA0_PAGE1_WORD5)	32	RW	FFFF_FFFFh
438h	Write Data (WDATA0_PAGE1_WORD6)	32	RW	FFFF_FFFFh
43Ch	Write Data (WDATA0_PAGE1_WORD7)	32	RW	FFFF_FFFFh
440h	Write Data (WDATA0_PAGE2_WORD0)	32	RW	FFFF_FFFFh
444h	Write Data (WDATA0_PAGE2_WORD1)	32	RW	FFFF_FFFFh
448h	Write Data (WDATA0_PAGE2_WORD2)	32	RW	FFFF_FFFFh
44Ch	Write Data (WDATA0_PAGE2_WORD3)	32	RW	FFFF_FFFFh
450h	Write Data (WDATA0_PAGE2_WORD4)	32	RW	FFFF_FFFFh

Table continues on the next page...

Table continued from the previous page...

Offset	Register	Width (In bits)	Access	Reset value
454h	Write Data (WDATA0_PAGE2_WORD5)	32	RW	FFFF_FFFFh
458h	Write Data (WDATA0_PAGE2_WORD6)	32	RW	FFFF_FFFFh
45Ch	Write Data (WDATA0_PAGE2_WORD7)	32	RW	FFFF_FFFFh
460h	Write Data (WDATA0_PAGE3_WORD0)	32	RW	FFFF_FFFFh
464h	Write Data (WDATA0_PAGE3_WORD1)	32	RW	FFFF_FFFFh
468h	Write Data (WDATA0_PAGE3_WORD2)	32	RW	FFFF_FFFFh
46Ch	Write Data (WDATA0_PAGE3_WORD3)	32	RW	FFFF_FFFFh
470h	Write Data (WDATA0_PAGE3_WORD4)	32	RW	FFFF_FFFFh
474h	Write Data (WDATA0_PAGE3_WORD5)	32	RW	FFFF_FFFFh
478h	Write Data (WDATA0_PAGE3_WORD6)	32	RW	FFFF_FFFFh
47Ch	Write Data (WDATA0_PAGE3_WORD7)	32	RW	FFFF_FFFFh
480h	Write Data (WDATA0_PAGE4_WORD0)	32	RW	FFFF_FFFFh
484h	Write Data (WDATA0_PAGE4_WORD1)	32	RW	FFFF_FFFFh
488h	Write Data (WDATA0_PAGE4_WORD2)	32	RW	FFFF_FFFFh
48Ch	Write Data (WDATA0_PAGE4_WORD3)	32	RW	FFFF_FFFFh
490h	Write Data (WDATA0_PAGE4_WORD4)	32	RW	FFFF_FFFFh
494h	Write Data (WDATA0_PAGE4_WORD5)	32	RW	FFFF_FFFFh
498h	Write Data (WDATA0_PAGE4_WORD6)	32	RW	FFFF_FFFFh
49Ch	Write Data (WDATA0_PAGE4_WORD7)	32	RW	FFFF_FFFFh
4A0h	Write Data (WDATA0_PAGE5_WORD0)	32	RW	FFFF_FFFFh
4A4h	Write Data (WDATA0_PAGE5_WORD1)	32	RW	FFFF_FFFFh
4A8h	Write Data (WDATA0_PAGE5_WORD2)	32	RW	FFFF_FFFFh
4ACh	Write Data (WDATA0_PAGE5_WORD3)	32	RW	FFFF_FFFFh
4B0h	Write Data (WDATA0_PAGE5_WORD4)	32	RW	FFFF_FFFFh
4B4h	Write Data (WDATA0_PAGE5_WORD5)	32	RW	FFFF_FFFFh
4B8h	Write Data (WDATA0_PAGE5_WORD6)	32	RW	FFFF_FFFFh
4BCh	Write Data (WDATA0_PAGE5_WORD7)	32	RW	FFFF_FFFFh
4C0h	Write Data (WDATA0_PAGE6_WORD0)	32	RW	FFFF_FFFFh
4C4h	Write Data (WDATA0_PAGE6_WORD1)	32	RW	FFFF_FFFFh
4C8h	Write Data (WDATA0_PAGE6_WORD2)	32	RW	FFFF_FFFFh
4CCh	Write Data (WDATA0_PAGE6_WORD3)	32	RW	FFFF_FFFFh

Table continues on the next page...

Table continued from the previous page...

Offset	Register	Width (In bits)	Access	Reset value
4D0h	Write Data (WDATA0_PAGE6_WORD4)	32	RW	FFFF_FFFFh
4D4h	Write Data (WDATA0_PAGE6_WORD5)	32	RW	FFFF_FFFFh
4D8h	Write Data (WDATA0_PAGE6_WORD6)	32	RW	FFFF_FFFFh
4DCh	Write Data (WDATA0_PAGE6_WORD7)	32	RW	FFFF_FFFFh
4E0h	Write Data (WDATA0_PAGE7_WORD0)	32	RW	FFFF_FFFFh
4E4h	Write Data (WDATA0_PAGE7_WORD1)	32	RW	FFFF_FFFFh
4E8h	Write Data (WDATA0_PAGE7_WORD2)	32	RW	FFFF_FFFFh
4ECh	Write Data (WDATA0_PAGE7_WORD3)	32	RW	FFFF_FFFFh
4F0h	Write Data (WDATA0_PAGE7_WORD4)	32	RW	FFFF_FFFFh
4F4h	Write Data (WDATA0_PAGE7_WORD5)	32	RW	FFFF_FFFFh
4F8h	Write Data (WDATA0_PAGE7_WORD6)	32	RW	FFFF_FFFFh
4FCh	Write Data (WDATA0_PAGE7_WORD7)	32	RW	FFFF_FFFFh
500h	Write Strobe (WSTRB0_PAGE0)	32	RW	FFFF_FFFFh
504h	Write Strobe (WSTRB0_PAGE1)	32	RW	FFFF_FFFFh
508h	Write Strobe (WSTRB0_PAGE2)	32	RW	FFFF_FFFFh
50Ch	Write Strobe (WSTRB0_PAGE3)	32	RW	FFFF_FFFFh
510h	Write Strobe (WSTRB0_PAGE4)	32	RW	FFFF_FFFFh
514h	Write Strobe (WSTRB0_PAGE5)	32	RW	FFFF_FFFFh
518h	Write Strobe (WSTRB0_PAGE6)	32	RW	FFFF_FFFFh
51Ch	Write Strobe (WSTRB0_PAGE7)	32	RW	FFFF_FFFFh
520h	Write Address (WADDR0)	32	RW	0000_0000h
524h	Write Control (WCTL0)	32	RW	0001_0000h
528h	Write Status (WSTAT0)	32	RW	001F_0000h
600h	Write Data (WDATA1_PAGE0_WORD0)	32	RW	FFFF_FFFFh
604h	Write Data (WDATA1_PAGE0_WORD1)	32	RW	FFFF_FFFFh
608h	Write Data (WDATA1_PAGE0_WORD2)	32	RW	FFFF_FFFFh
60Ch	Write Data (WDATA1_PAGE0_WORD3)	32	RW	FFFF_FFFFh
610h	Write Data (WDATA1_PAGE0_WORD4)	32	RW	FFFF_FFFFh
614h	Write Data (WDATA1_PAGE0_WORD5)	32	RW	FFFF_FFFFh
618h	Write Data (WDATA1_PAGE0_WORD6)	32	RW	FFFF_FFFFh
61Ch	Write Data (WDATA1_PAGE0_WORD7)	32	RW	FFFF_FFFFh

Table continues on the next page...

Table continued from the previous page...

Offset	Register	Width (In bits)	Access	Reset value
620h	Write Data (WDATA1_PAGE1_WORD0)	32	RW	FFFF_FFFFh
624h	Write Data (WDATA1_PAGE1_WORD1)	32	RW	FFFF_FFFFh
628h	Write Data (WDATA1_PAGE1_WORD2)	32	RW	FFFF_FFFFh
62Ch	Write Data (WDATA1_PAGE1_WORD3)	32	RW	FFFF_FFFFh
630h	Write Data (WDATA1_PAGE1_WORD4)	32	RW	FFFF_FFFFh
634h	Write Data (WDATA1_PAGE1_WORD5)	32	RW	FFFF_FFFFh
638h	Write Data (WDATA1_PAGE1_WORD6)	32	RW	FFFF_FFFFh
63Ch	Write Data (WDATA1_PAGE1_WORD7)	32	RW	FFFF_FFFFh
640h	Write Data (WDATA1_PAGE2_WORD0)	32	RW	FFFF_FFFFh
644h	Write Data (WDATA1_PAGE2_WORD1)	32	RW	FFFF_FFFFh
648h	Write Data (WDATA1_PAGE2_WORD2)	32	RW	FFFF_FFFFh
64Ch	Write Data (WDATA1_PAGE2_WORD3)	32	RW	FFFF_FFFFh
650h	Write Data (WDATA1_PAGE2_WORD4)	32	RW	FFFF_FFFFh
654h	Write Data (WDATA1_PAGE2_WORD5)	32	RW	FFFF_FFFFh
658h	Write Data (WDATA1_PAGE2_WORD6)	32	RW	FFFF_FFFFh
65Ch	Write Data (WDATA1_PAGE2_WORD7)	32	RW	FFFF_FFFFh
660h	Write Data (WDATA1_PAGE3_WORD0)	32	RW	FFFF_FFFFh
664h	Write Data (WDATA1_PAGE3_WORD1)	32	RW	FFFF_FFFFh
668h	Write Data (WDATA1_PAGE3_WORD2)	32	RW	FFFF_FFFFh
66Ch	Write Data (WDATA1_PAGE3_WORD3)	32	RW	FFFF_FFFFh
670h	Write Data (WDATA1_PAGE3_WORD4)	32	RW	FFFF_FFFFh
674h	Write Data (WDATA1_PAGE3_WORD5)	32	RW	FFFF_FFFFh
678h	Write Data (WDATA1_PAGE3_WORD6)	32	RW	FFFF_FFFFh
67Ch	Write Data (WDATA1_PAGE3_WORD7)	32	RW	FFFF_FFFFh
680h	Write Data (WDATA1_PAGE4_WORD0)	32	RW	FFFF_FFFFh
684h	Write Data (WDATA1_PAGE4_WORD1)	32	RW	FFFF_FFFFh
688h	Write Data (WDATA1_PAGE4_WORD2)	32	RW	FFFF_FFFFh
68Ch	Write Data (WDATA1_PAGE4_WORD3)	32	RW	FFFF_FFFFh
690h	Write Data (WDATA1_PAGE4_WORD4)	32	RW	FFFF_FFFFh
694h	Write Data (WDATA1_PAGE4_WORD5)	32	RW	FFFF_FFFFh
698h	Write Data (WDATA1_PAGE4_WORD6)	32	RW	FFFF_FFFFh

Table continues on the next page...

Table continued from the previous page...

Offset	Register	Width (In bits)	Access	Reset value
69Ch	Write Data (WDATA1_PAGE4_WORD7)	32	RW	FFFF_FFFFh
6A0h	Write Data (WDATA1_PAGE5_WORD0)	32	RW	FFFF_FFFFh
6A4h	Write Data (WDATA1_PAGE5_WORD1)	32	RW	FFFF_FFFFh
6A8h	Write Data (WDATA1_PAGE5_WORD2)	32	RW	FFFF_FFFFh
6ACh	Write Data (WDATA1_PAGE5_WORD3)	32	RW	FFFF_FFFFh
6B0h	Write Data (WDATA1_PAGE5_WORD4)	32	RW	FFFF_FFFFh
6B4h	Write Data (WDATA1_PAGE5_WORD5)	32	RW	FFFF_FFFFh
6B8h	Write Data (WDATA1_PAGE5_WORD6)	32	RW	FFFF_FFFFh
6BCh	Write Data (WDATA1_PAGE5_WORD7)	32	RW	FFFF_FFFFh
6C0h	Write Data (WDATA1_PAGE6_WORD0)	32	RW	FFFF_FFFFh
6C4h	Write Data (WDATA1_PAGE6_WORD1)	32	RW	FFFF_FFFFh
6C8h	Write Data (WDATA1_PAGE6_WORD2)	32	RW	FFFF_FFFFh
6CCh	Write Data (WDATA1_PAGE6_WORD3)	32	RW	FFFF_FFFFh
6D0h	Write Data (WDATA1_PAGE6_WORD4)	32	RW	FFFF_FFFFh
6D4h	Write Data (WDATA1_PAGE6_WORD5)	32	RW	FFFF_FFFFh
6D8h	Write Data (WDATA1_PAGE6_WORD6)	32	RW	FFFF_FFFFh
6DCh	Write Data (WDATA1_PAGE6_WORD7)	32	RW	FFFF_FFFFh
6E0h	Write Data (WDATA1_PAGE7_WORD0)	32	RW	FFFF_FFFFh
6E4h	Write Data (WDATA1_PAGE7_WORD1)	32	RW	FFFF_FFFFh
6E8h	Write Data (WDATA1_PAGE7_WORD2)	32	RW	FFFF_FFFFh
6ECh	Write Data (WDATA1_PAGE7_WORD3)	32	RW	FFFF_FFFFh
6F0h	Write Data (WDATA1_PAGE7_WORD4)	32	RW	FFFF_FFFFh
6F4h	Write Data (WDATA1_PAGE7_WORD5)	32	RW	FFFF_FFFFh
6F8h	Write Data (WDATA1_PAGE7_WORD6)	32	RW	FFFF_FFFFh
6FCh	Write Data (WDATA1_PAGE7_WORD7)	32	RW	FFFF_FFFFh
700h	Write Strobe (WSTRB1_PAGE0)	32	RW	FFFF_FFFFh
704h	Write Strobe (WSTRB1_PAGE1)	32	RW	FFFF_FFFFh
708h	Write Strobe (WSTRB1_PAGE2)	32	RW	FFFF_FFFFh
70Ch	Write Strobe (WSTRB1_PAGE3)	32	RW	FFFF_FFFFh
710h	Write Strobe (WSTRB1_PAGE4)	32	RW	FFFF_FFFFh
714h	Write Strobe (WSTRB1_PAGE5)	32	RW	FFFF_FFFFh

Table continues on the next page...

Table continued from the previous page...

Offset	Register	Width (In bits)	Access	Reset value
718h	Write Strobe (WSTRB1_PAGE6)	32	RW	FFFF_FFFFh
71Ch	Write Strobe (WSTRB1_PAGE7)	32	RW	FFFF_FFFFh
720h	Write Address (WADDR1)	32	RW	0000_0000h
724h	Write Control (WCTL1)	32	RW	0001_0000h
728h	Write Status (WSTAT1)	32	RW	001F_0000h
FF0h	Debug Status 0 (DBGSTAT0)	32	R	0000_0000h
FF4h	Debug Status 1 (DBGSTAT1)	32	R	0000_00FFh
FF8h	Debug Status 2 (DBGSTAT2)	32	R	8000_0001h
FFCh	Debug Status 3 (DBGSTAT3)	32	R	003F_FFFFh

7.2 Port Configuration (PC1 - PC3)

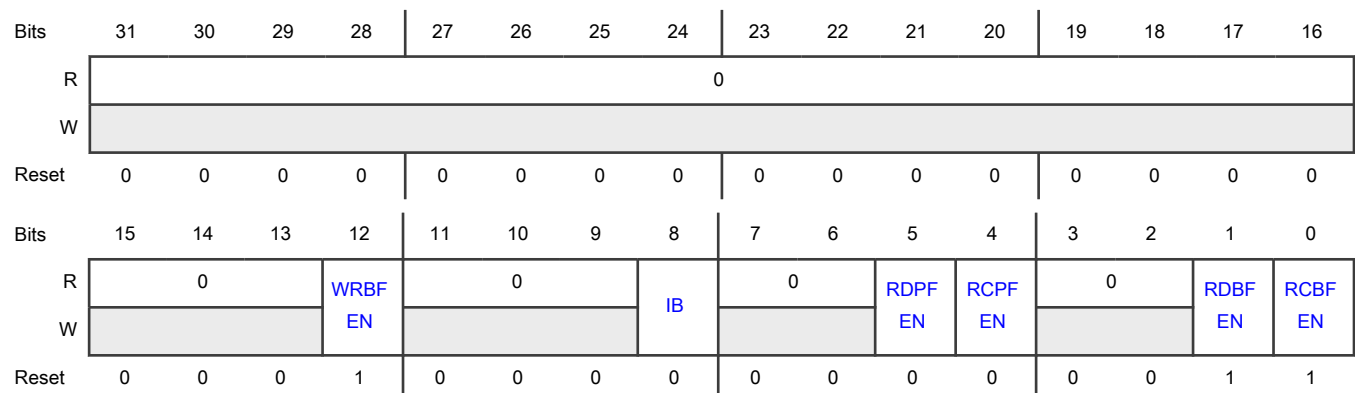
Offset

Register	Offset
PC1	4h
PC2	8h
PC3	Ch

Function

Specifies the configuration of MRAMC system port n.

Diagram



Fields

Field	Function
31-13 —	Reserved
12 WRBFEN	Write Buffer Enable Enables write buffer. When enabled, write transactions hits in write buffer are merged. 0b - Disable. It is not recommended to use this value. 1b - Enable
11-9 —	Reserved
8 IB	Invalidate Buffer Invalidates all read buffer entries and flush all write buffer entries for this port. This field is set to 0 by hardware once operation is completed. 0b - No invalidation 1b - Invalidate
7-6 —	Reserved
5 RDPFEN	Read Data Prefetch Enable Enables or disables data prefetching initiated by a read access on this Port. Prefetching can only be enabled if the buffers are enabled by writing 1 to RDBFEN . Hardware reset returns this field to 0. 0b - Disable 1b - Enable
4 RCPFEN	Read Code Prefetch Enable Enables or disables code prefetching initiated by a read access on this Port. Prefetching can only be enabled if the buffers are enabled by writing 1 to RCBFEN . Hardware reset returns this field to 0. 0b - Disable 1b - Enable
3-2 —	Reserved
1 RDBFEN	Read Data Buffers Enable Enables or disables line read data buffer hits. If this field is 0, the line read buffers are disabled from satisfying read requests, and all buffer valid bits are set to 0. If this field is enabled, the line read buffers are enabled to satisfy read requests on hits. Buffer valid bits may be set when the buffers are successfully filled.

Table continues on the next page...

Table continued from the previous page...

Field	Function
	0b - Disable 1b - Enable
0 RCBFEN	Read Code Buffers Enable Enables or disables line read code buffer hits. If disabled, the line read buffers are disabled from satisfying read requests, and all buffer valid bits are set to 0. If enabled, the line read buffers are enabled to satisfy read requests on hits. Buffer valid bits may be set when the buffers are successfully filled. 0b - Disable 1b - Enable

7.3 Port i Read ID Assignment Word j (P1_RIA_W0 - P2_RIA_W1)

Offset

Register	Offset
P1_RIA_W0	90h
P1_RIA_W1	94h
P2_RIA_W0	A0h
P2_RIA_W1	A4h

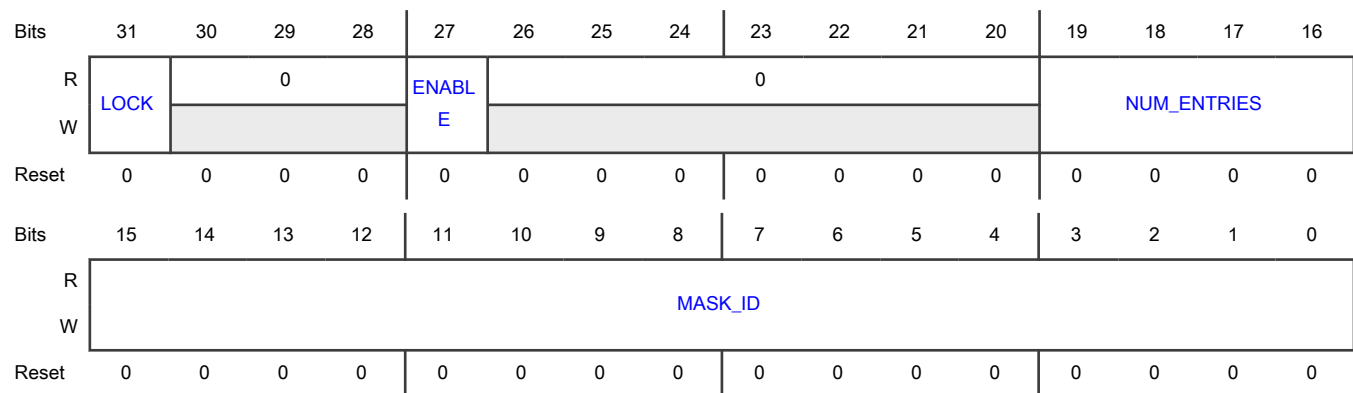
Function

Port i Read ID Assignment Word j.

NOTE

Multiple registers for specific port must not have overlapped assignments for a DID, otherwise it results in register bus transfer error while programming. Multiple registers for specific port must not have NUM_ENTRIES field configured such that their addition is greater than total entries in buffer. These registers must only be updated after the read buffer has been invalidated. If a DID does not belong to any DID groups, the access is treated as non-cacheable.

Diagram



Fields

Field	Function
31 LOCK	Register Lock 0b - Write access to the register is allowed. 1b - Write access to the register is not allowed until next reset.
30-28 —	Reserved
27 ENABLE	Enable 0b - Disabled. 1b - Enabled.
26-20 —	Reserved
19-16 NUM_ENTRIES	Number of Buffer Entries Plus 1
15-0 MASK_ID	Mask of Domain IDs The position of each bit equals to one selects one ID. For example: the value 0x0001 selects the ID 0; the value 0x0003 selects the ID 0 and 1; the value 0x0007 selects the IDs 0, 1 and 2; and so on.

7.4 Port i Read ID Assignment Word j (P3_RIA_W0)

Offset

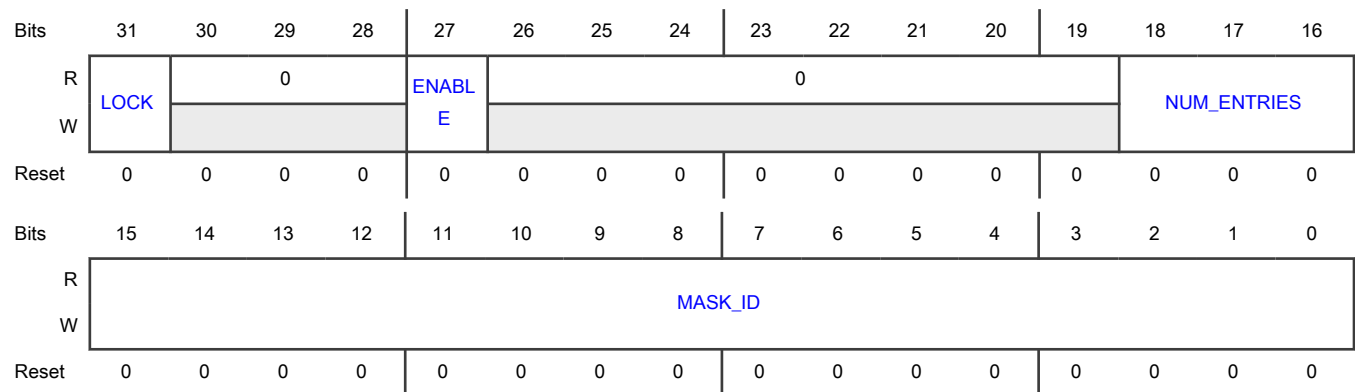
Register	Offset
P3_RIA_W0	B0h

Function

Port i Read ID Assignment Word j.

NOTE

Multiple registers for specific port must not have overlapped assignments for a DID, otherwise it results in register bus transfer error while programming. Multiple registers for specific port must not have NUM_ENTRIES field configured such that their addition is greater than total entries in buffer. These registers must only be updated after the read buffer has been invalidated. If a DID does not belong to any DID groups, the access is treated as non-cacheable.

Diagram**Fields**

Field	Function
31 LOCK	Register Lock 0b - Write access to the register is allowed. 1b - Write access to the register is not allowed until next reset.
30-28 —	Reserved
27 ENABLE	Enable 0b - Disabled. 1b - Enabled.
26-19 —	Reserved
18-16 NUM_ENTRIES	Number of Buffer Entries Plus 1
15-0 MASK_ID	Mask of Domain IDs The position of each bit equals to one selects one ID. For example: the value 0x0001 selects the ID 0; the value 0x0003 selects the ID 0 and 1; the value 0x0007 selects the IDs 0, 1 and 2; and so on.

7.5 Large Partition Lock (LPLOCK)

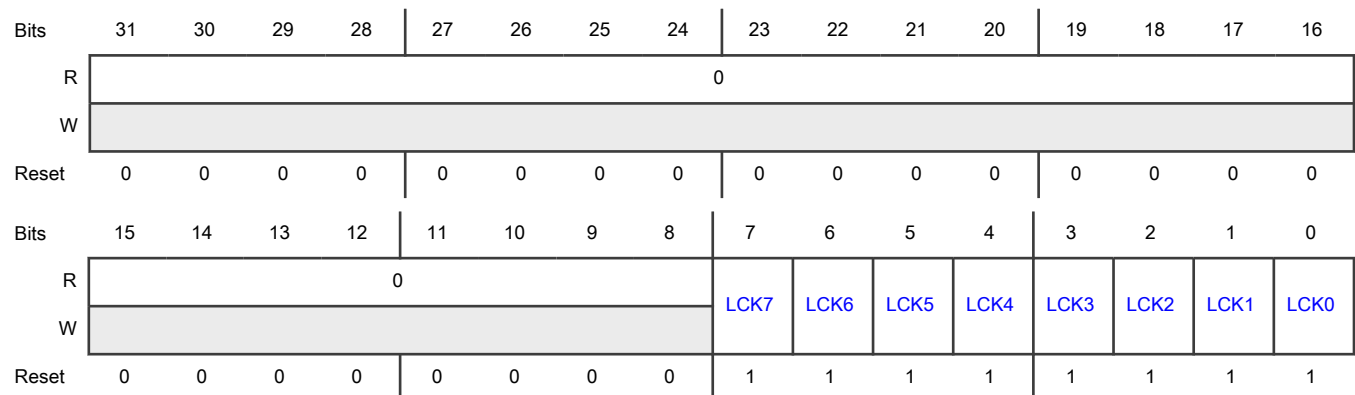
Offset

Register	Offset
LPLOCK	100h

Function

Provides a way to protect large partition from being modified.

Diagram



Fields

Field	Function
31-8 —	Reserved
7-0 LCKm	Lock Locks selected partition pair (upper and lower partitions). 0b - The partition pair is available for write 1b - The partition pair is not available for write

7.6 Small Partition Lock (SPLOCK)

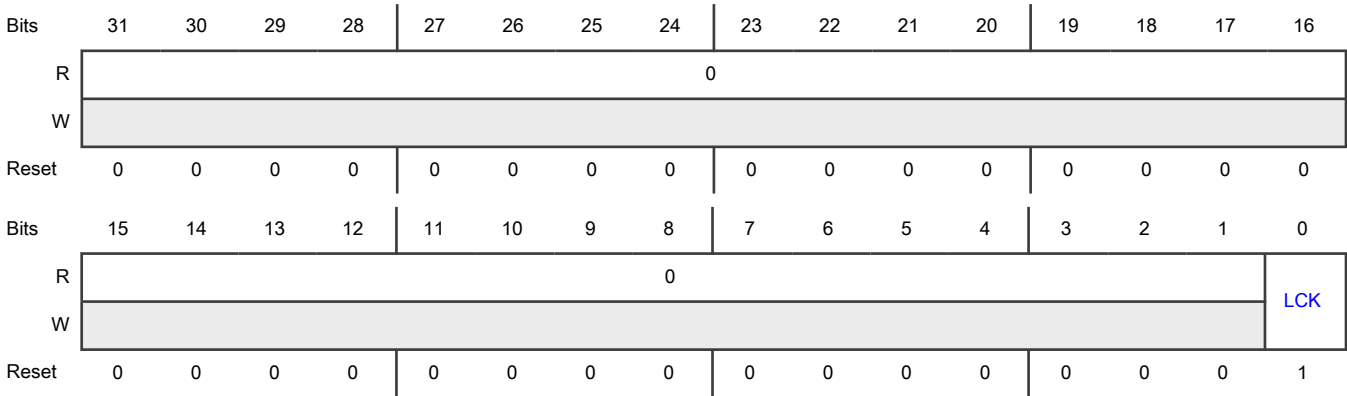
Offset

Register	Offset
SPLOCK	110h

Function

Provides a way to protect small partition from being modified.

Diagram



Fields

Field	Function
31-1 —	Reserved
0 LCK	Lock Locks selected partition. 0b - The partition is available for write 1b - The partition is not available for write

7.7 Large Partition OTP Lock (LPOLOCK)

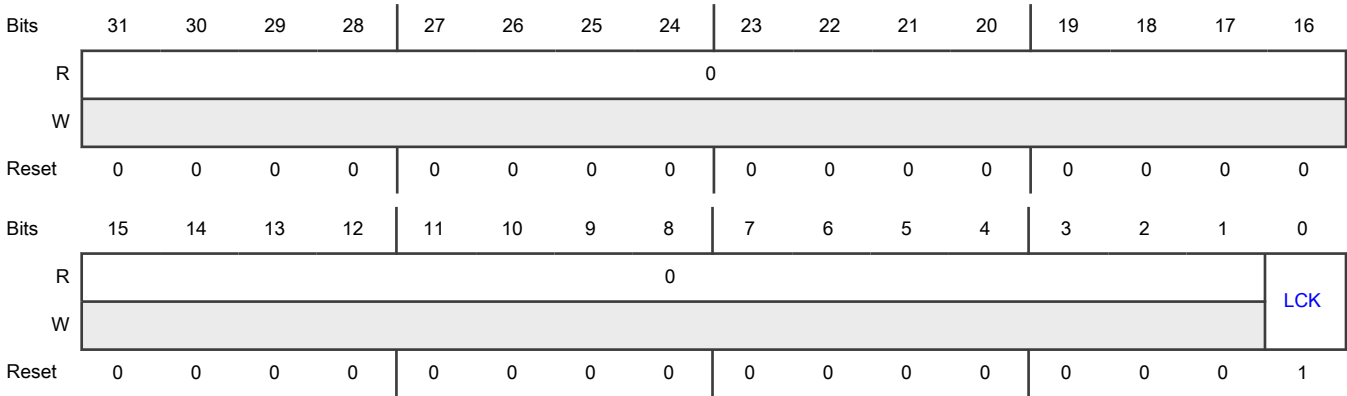
Offset

Register	Offset
LPOLOCK	118h

Function

Provides a way to protect large partition OTP space from being modified.

Diagram



Fields

Field	Function
31-1 —	Reserved
0 LCK	Lock Locks selected partition. NOTE There is single lock control for all large partitions' OTP space. 0b - The partition is available for write 1b - The partition is not available for write

7.8 UTest Lock (UTLOCK)

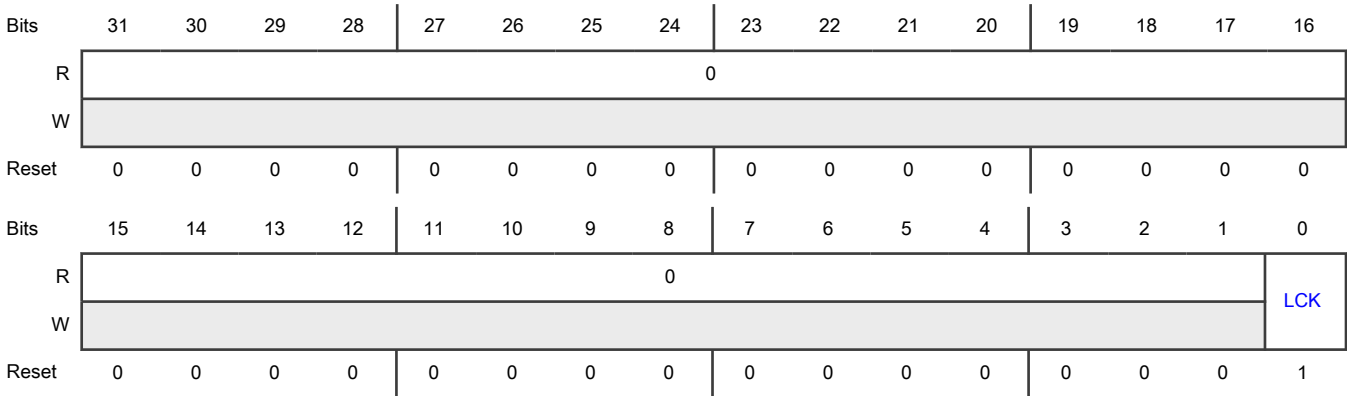
Offset

Register	Offset
UTLOCK	11Ch

Function

Provides a way to protect UTest space from being modified.

Diagram



Fields

Field	Function
31-1 —	Reserved
0 LCK	Lock Locks selected partition. NOTE There is single lock control for all small and large partitions' UTest space. 0b - The partition is available for write 1b - The partition is not available for write

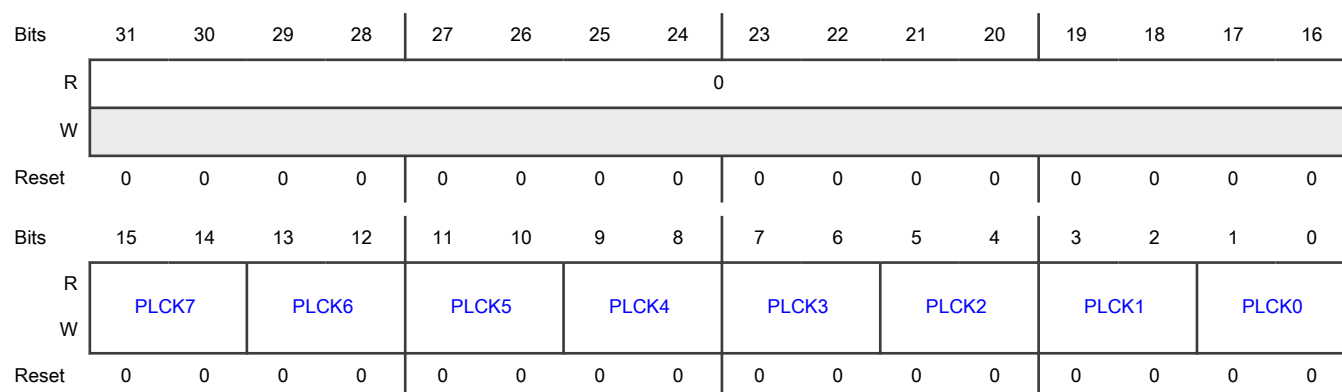
7.9 Protect Large Partition Lock Register (PLPLOCK0)

Offset

Register	Offset
PLPLOCK0	130h

Function

Provides a way to protect bits in LPLOCK.

Diagram**Fields**

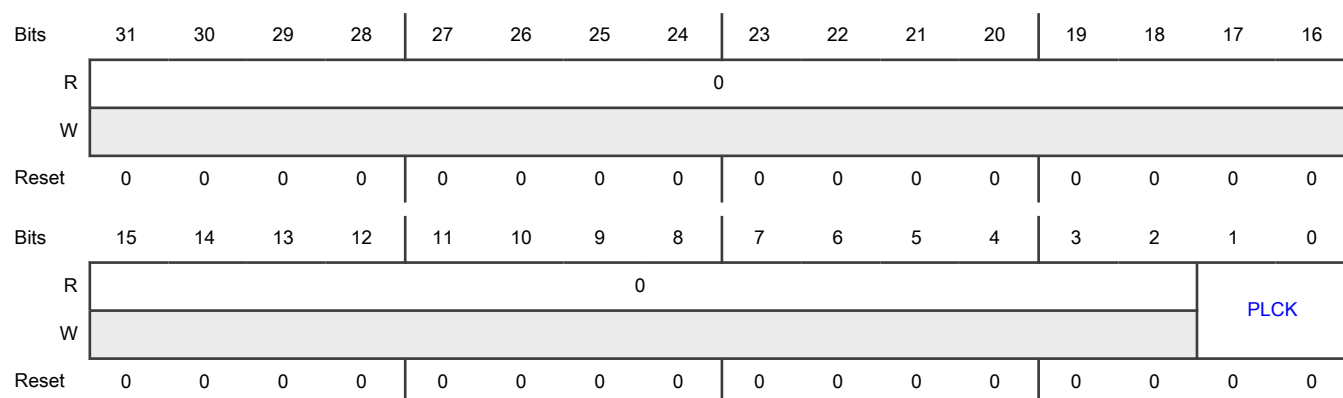
Field	Function
31-16 —	Reserved
15-14: PLCK7 13-12: PLCK6 11-10: PLCK5 9-8: PLCK4 7-6: PLCK3 5-4: PLCK2 3-2: PLCK1 1-0: PLCK0	Protect Lock Register Protects corresponding LPLOCK bit. 00b - LPLOCK bit can be set and cleared by any initiator. 01b - LPLOCK bit can be set and cleared by any initiator. 10b - LPLOCK bit can be set and cleared only by initiator with domain ID that owns LPLOCK bit. Initiator can try to gain ownership of corresponding LPLOCK bit based on domain ID by writing value 10b to this field if not already set to value 10b. Once this field is set to 10b, only that domain ID can change LPLOCK bit value. If LPLOCK bit is already acquired by a particular domain ID, any effort to modify it by initiator with a different domain ID is ignored without transaction transfer error. Lock register bit ownership is not valid when this field has value other than 10b. After this field is set to 10b, it can only be changed to 11b by owning domain ID, or to 00b or 01b on next reset. 11b - LPLOCK bit can not be cleared once set. After this field set to 11b, it can not be changed until next reset.

7.10 Protect Small Partition Lock Register (PSPLOCK)**Offset**

Register	Offset
PSPLOCK	140h

Function

Provides a way to protect bits in SPLOCK.

Diagram**Fields**

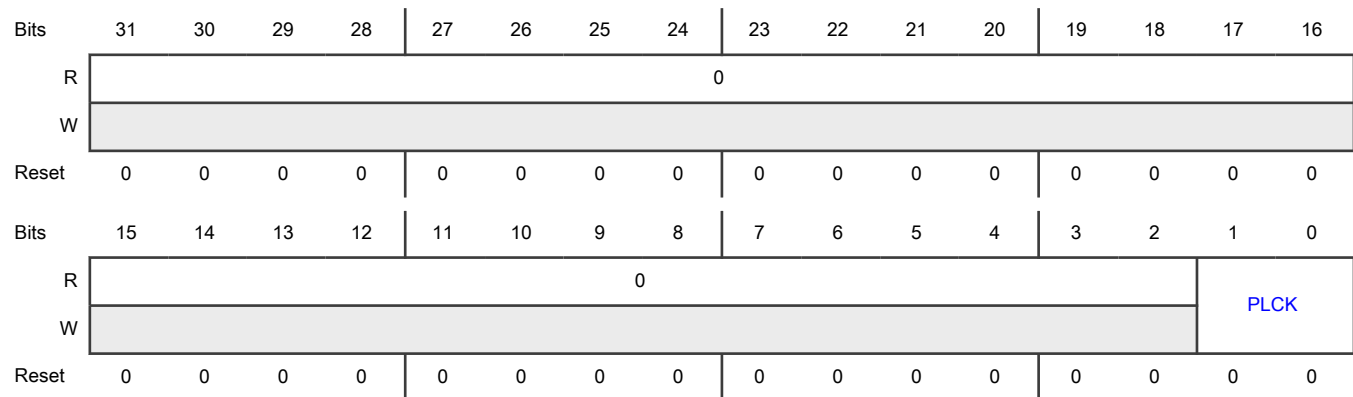
Field	Function
31-2 —	Reserved
1-0 PLCK	Protect Lock register Protects SPLOCK. 00b - SPLOCK bit can be set and cleared by any initiator. 01b - SPLOCK bit can be set and cleared by any initiator. 10b - SPLOCK bit can be set and cleared only by initiator with domain ID that owns SPLOCK bit. Initiator can try to gain ownership of corresponding SPLOCK bit based on domain ID by writing value 10b to this field if not already set to value 10b. Once this field is set to 10b, only that domain ID can change SPLOCK bit value. If SPLOCK bit is already acquired by a particular domain ID, any effort to modify it by initiator with a different domain ID is ignored without transaction transfer error. Lock register bit ownership is not valid when this field has value other than 10b. After this field is set to 10b, it can only be changed to 11b by owning domain ID, or to 00b or 01b on next reset. 11b - SPLOCK bit can not be cleared once set. After this field set to 11b, it can not be changed until next reset.

7.11 Protect Large Partition OTP Lock Register (PLPOLOCK)**Offset**

Register	Offset
PLPOLOCK	148h

Function

Provides a way to protect bits in LPOLOCK.

Diagram**Fields**

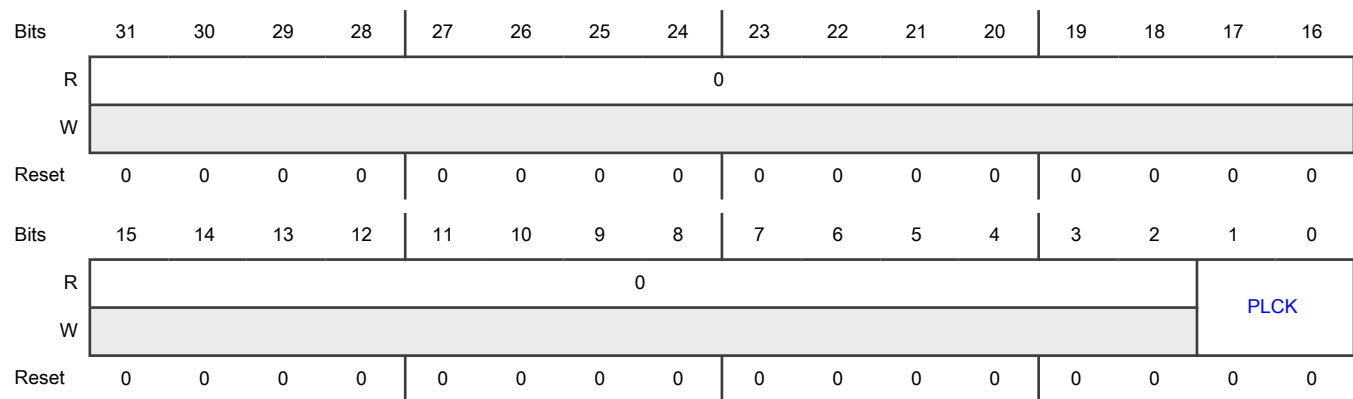
Field	Function
31-2 —	Reserved
1-0 PLCK	Protect Lock Register Protects LPOLOCK. 00b - LPOLOCK bit can be set and cleared by any initiator. 01b - LPOLOCK bit can be set and cleared by any initiator. 10b - LPOLOCK bit can be set and cleared only by initiator with domain ID that owns LPOLOCK bit. Initiator can try to gain ownership of corresponding LPOLOCK bit based on domain ID by writing value 10b to this field if not already set to value 10b. Once this field is set to 10b, only that domain ID can change LPOLOCK bit value. If LPOLOCK bit is already acquired by a particular domain ID, any effort to modify it by initiator with a different domain ID is ignored without transaction transfer error. Lock register bit ownership is not valid when this field has value other than 10b. After this field is set to 10b, it can only be changed to 11b by owning domain ID, or to 00b or 01b on next reset. 11b - LPOLOCK bit can not be cleared once set. After this field set to 11b, it can not be changed until next reset.

7.12 Protect UTest Lock Register (PUTLOCK)**Offset**

Register	Offset
PUTLOCK	14Ch

Function

Provides a way to protect bits in UTLOCK.

Diagram**Fields**

Field	Function
31-2 —	Reserved
1-0 PLCK	Protect Lock Register Protects UTLOCK. 00b - UTLOCK bit can be set and cleared by any initiator. 01b - UTLOCK bit can be set and cleared by any initiator. 10b - UTLOCK bit can be set and cleared only by initiator with domain ID that owns UTest lock register bit. Initiator can try to gain ownership of corresponding UTLOCK bit based on domain ID by writing value 10b to this field if not already set to value 10b. Once this field is set to 10b, only that domain ID can change UTLOCK bit value. If UTLOCK bit is already acquired by a particular domain ID, any effort to modify it by initiator with a different domain ID is ignored without transaction transfer error. Lock register bit ownership is not valid when this field has value other than 10b. After this field is set to 10b, it can only be changed to 11b by owning domain ID, or to 00b or 01b on next reset. 11b - UTLOCK bit can not be cleared once set. After this field set to 11b, it can not be changed until next reset.

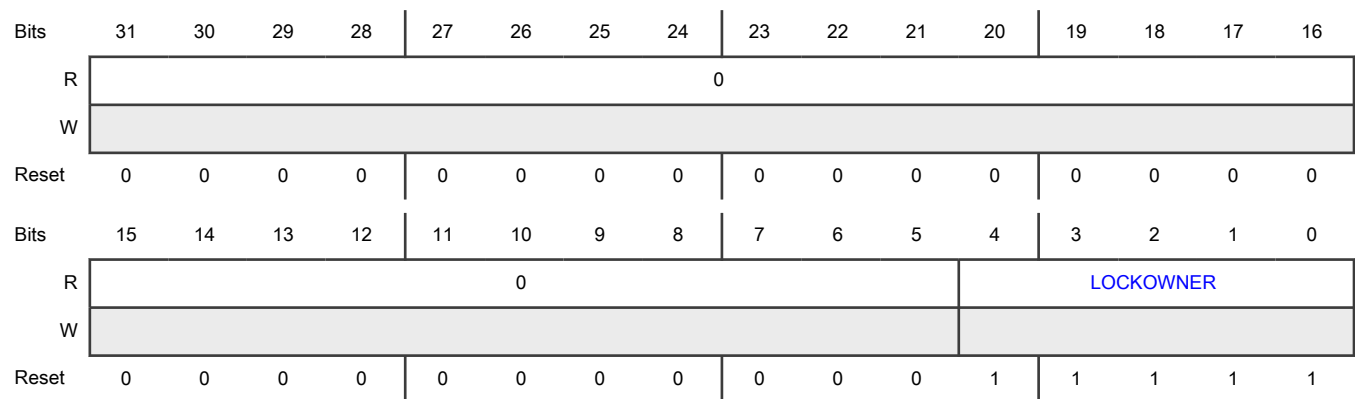
7.13 Large Partition Lock Owner (LPLOCKOW0 - LPLOCKOW7)**Offset**

For n = 0 to 7:

Register	Offset
LPLOCKOWn	200h + (n × 4h)

Function

Provides the domain ID information of the initiator currently acquiring the corresponding lock register bit.

Diagram**Fields**

Field	Function
31-5 —	Reserved
4-0 LOCKOWNER	Lock Owner Contains domain ID of the initiator currently acquiring the corresponding lock register bit. Value 5'd31b represents that none of the initiator has ownership of the corresponding lock register bit. Lock register bit ownership is not valid when corresponding protect lock register field has value other than 10b.

7.14 Partition Lock Owner (SPLOCKOW - UTLOCKOW)**Offset**

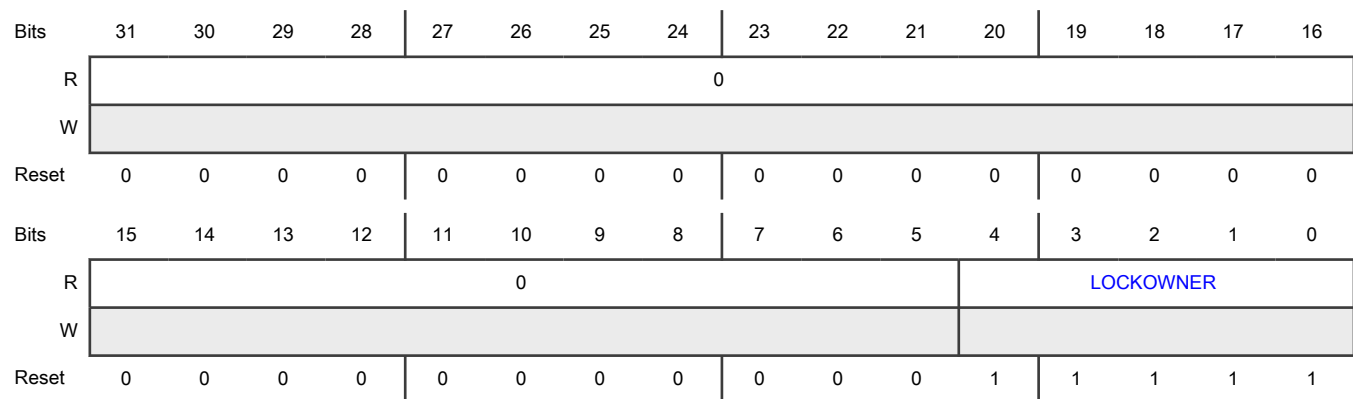
Register	Offset
SPLOCKOW	300h
LPOLOCKOW	304h
UTLOCKOW	308h

Function

Provides the domain ID information of the initiator currently acquiring the corresponding lock register bit.

Register acronym:

- SPLOCKOW : Small partition lock owner
- LPOLOCKOW: Large partition OTP lock owner
- UTLOCKOW : UTest lock owner

Diagram**Fields**

Field	Function
31-5 —	Reserved
4-0 LOCKOWNER	Lock Owner Contains domain ID of the initiator currently acquiring the corresponding lock register bit. Value 5'd31b represents that none of the initiator has ownership of the corresponding lock register bit. Lock register bit ownership is not valid when corresponding protect lock register field has value other than 10b.

7.15 Error Injection Control (ERRINJCTL)**Offset**

Register	Offset
ERRINJCTL	380h

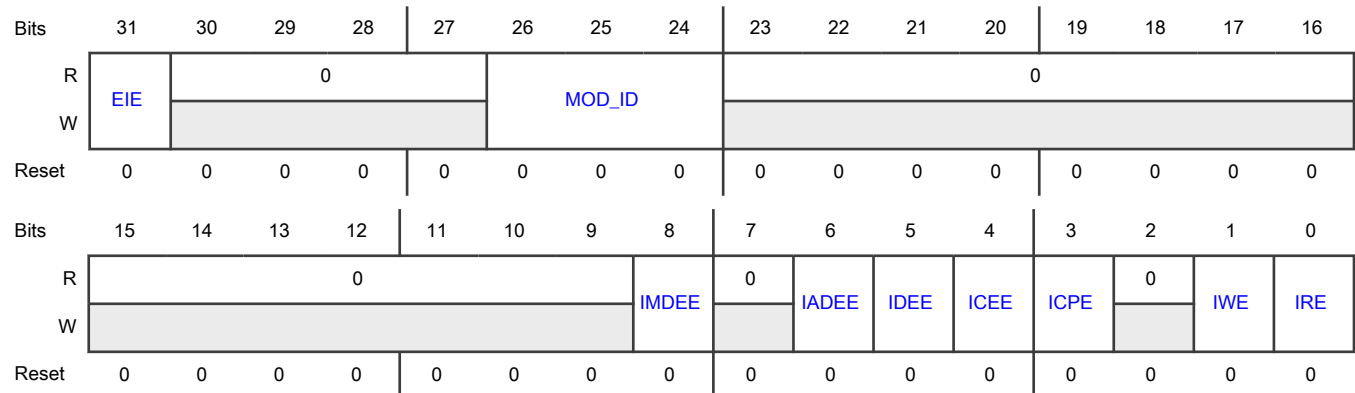
Function

Provides a way to inject error in safety logic to test safety mechanism. Using field in this register to inject error will set corresponding field in ERRSTAT0 register once error is detected. For example, ERRINJCTL[ICPE] will set ERRSTAT0[CPE]. Steps to inject and check error are listed below.

- Set EIE to 1.
- Set MOD_ID to desired module.
- Set IWE or IRE to 1 to select write or read path.
- Select one type of error to inject in this register and set it to 1.
- Above steps can be clubbed together in single write.
- Initiate write or read transaction based on IWE or IRE selected above.
- Wait for corresponding error to be set in ERRSTAT0 register.
- Check fault to be set in FCCU if needed.

- Clear selected error injection type in this register.
- Clear error in ERRSTAT0 register.
- Repeat above steps to iterate through other errors if needed.
- Write value 0 in this register to disable error injection.

Diagram



Fields

Field	Function
31 EIE	Error Injection Enable Injects error in safety logic. This bit need to be set to 1 to enable error injection using other fields in this register. 0b - No error injection. 1b - Inject error.
30-27 —	Reserved
26-24 MOD_ID	Module ID Selects the module to inject error to. 000b - Register bus interface. Only IADEE is applicable for this MOD_ID. 001b - Reserved 010b - Reserved 011b - System bus interface 1 100b - System bus interface 2 101b - System bus interface 3 110b - Reserved 111b - MRAM interface
23-9	Reserved

Table continues on the next page...

Table continued from the previous page...

Field	Function
—	
8 IMDEE	Inject MRAM Data ECC Error Injects ECC error on read data from MRAM that has different ECC scheme than MRAMC internal ECC. This sets error in ERRSTAT0[DEE] once error is detected. This field is not applicable for MOD_ID value of register bus interface or system bus interface. 0b - Do not inject error. 1b - Inject error.
7 —	Reserved
6 IADEE	Inject Address Decode Error Injects error on address decoder. 0b - Do not inject error. 1b - Inject error.
5 IDEE	Inject Data ECC Error Injects ECC error on read data. 0b - Do not inject error. 1b - Inject error.
4 ICEE	Inject Control ECC Error Injects ECC error on control attributes like address. 0b - Do not inject error. 1b - Inject error.
3 ICPE	Inject Control Parity Error Injects parity error on control attributes. 0b - Do not inject error. 1b - Inject error.
2 —	Reserved
1 IWE	Inject Write Error Injects error on write path. Either this field or IRE need to be set to enable any error injection listed in this register. 0b - Do not inject error. 1b - Inject error.

Table continues on the next page...

Table continued from the previous page...

Field	Function
0	Inject Read Error
IRE	Injects error on read path. Either this field or IWE need to be set to enable any error injection listed in this register. 0b - Do not inject error. 1b - Inject error.

7.16 Error Status 0 (ERRSTAT0)

Offset

Register	Offset
ERRSTAT0	388h

Function

Reports status of error encountered.

Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															MEEE
W																W1C
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0								ME	MCE	DEE	CEE	CPE	PWE	WE	RE
W									W1C	W1C	W1C	W1C	W1C	W1C	W1C	W1C
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fields

Field	Function
31-17 —	Reserved
16 MEEE	MRAM EDC After ECC Error Encounters EDC after ECC error from MRAM, see MRAM MCRS[EEE]. Address captured in ERRSTAT1 register is not valid for this error. This error raises FCCU alarm.

Table continues on the next page...

Table continued from the previous page...

Field	Function
	0b - No error detected. 1b - Error detected.
15-8 —	Reserved
7 ME	MRAM Error Encounters error from MRAM for any of below conditions. • Write verify error for write, see MRAM MCRS[WRE]. • Uncorrectable read data ECC error for read, see MRAM MCRS[EER]. • Address encode error for read, see MRAM MCRS[AEE]. This error raises FCCU alarm. 0b - No error detected. 1b - Error detected.
6 MCE	MRAM controller error Encounters Error with MRAMC address decoder logic. This error raises interrupt. 0b - No error detected. 1b - Error detected.
5 DEE	Data ECC Error Encounters ECC error on read data. This error raises FCCU alarm. 0b - No error detected. 1b - Error detected.
4 CEE	Control ECC Error Encounters ECC error on control attributes like address for any of below conditions. • ECC error on address on request path from system bus port that supports ECC on bus to MRAMC boundary. • ECC error on address on request path from MRAMC AXI boundary to address decoder in MRAMC. • ECC error on address on request path from address decoder in MRAMC to MRAM logic boundary in MRAMC. This error raises FCCU alarm. 0b - No error detected. 1b - Error detected.

Table continues on the next page...

Table continued from the previous page...

Field	Function
3 CPE	Control Parity Error Encounters parity error on control attributes on system bus interface to MRAMC boundary. Only AXI valid and ready are protected with parity. This error raises FCCU alarm. 0b - No error detected. 1b - Error detected.
2 PWE	Posted Write Error Encounters error on posted (or bufferable) write transaction. This error raises interrupt. 0b - No error detected. 1b - Error detected.
1 WE	Write Error Encounters error on write path. See other fields in this register for exact error. 0b - No error detected. 1b - Error detected.
0 RE	Read Error Encounters error on read path. See other fields in this register for exact error. 0b - No error detected. 1b - Error detected.

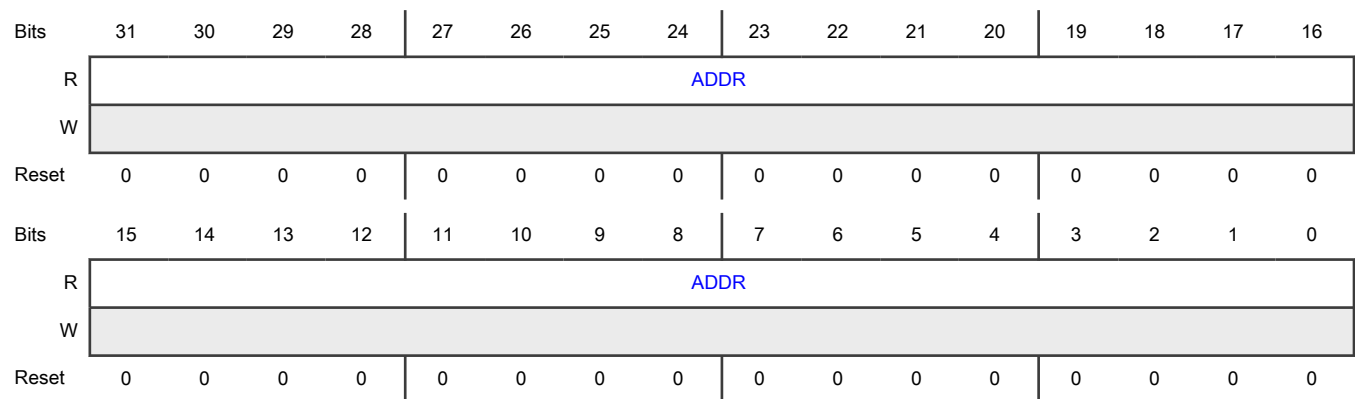
7.17 Error Status 1 (ERRSTAT1)

Offset

Register	Offset
ERRSTAT1	38Ch

Function

Reports address of the transaction that encountered error.

Diagram**Fields**

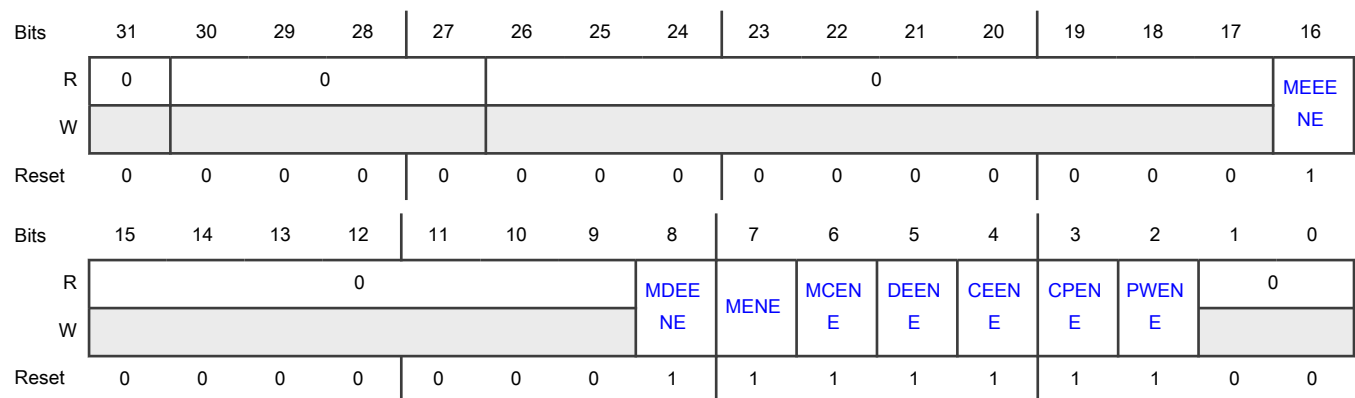
Field	Function
31-0	Address of Error
ADDR	Address of transaction that encountered error. This is sticky field that captures first error and ignores update for later errors, until all error bits in ERRSTAT0 are cleared by your software.

7.18 Error Notification Control 0 (ERRNTCTL0)**Offset**

Register	Offset
ERRNTCTL0	3A0h

Function

Provides a way to control error notification to system. The fields in this register do not affect set or reset status in ERRSTAT0, but it controls ERRSTAT0 notification to the system.

Diagram

Fields

Field	Function
31 —	Reserved
30-27 —	Reserved
26-17 —	Reserved
16 MEEENE	MRAM EDC After ECC Error Notification Enable Enables ERRSTAT0[MEEE] to be notified to FCCU. 0b - Notification is disabled. 1b - Notification is enabled.
15-9 —	Reserved
8 MDEENE	MRAM Data ECC Error Notification Enable Enables ERRSTAT0[MDEE] to be notified to FCCU. 0b - Notification is disabled. 1b - Notification is enabled.
7 MENE	MRAM Error Notification Enable Enables ERRSTAT0[ME] to be notified to FCCU. 0b - Notification is disabled. 1b - Notification is enabled.
6 MCENE	MRAMC Error Notification Enable Enables ERRSTAT0[MCE] to be notified to interrupt controller. 0b - Notification is disabled. 1b - Notification is enabled.
5 DEENE	Data ECC Error Notification Enable Enables ERRSTAT0[DEE] to be notified to FCCU. 0b - Notification is disabled. 1b - Notification is enabled.
4 CEENE	Control ECC Error Notification Enable Enables ERRSTAT0[CEE] to be notified to FCCU.

Table continues on the next page...

Table continued from the previous page...

Field	Function
	0b - Notification is disabled. 1b - Notification is enabled.
3 CPENE	Control Parity Error Notification Enable Enables ERRSTAT0[CPE] to be notified to FCCU. 0b - Notification is disabled. 1b - Notification is enabled.
2 PWENE	Posted Write Error Notification Enable Enables ERRSTAT0[PWE] to be notified to interrupt controller. 0b - Notification is disabled. 1b - Notification is enabled.
1-0 —	Reserved

7.19 Write Watchdog (WWD0G)

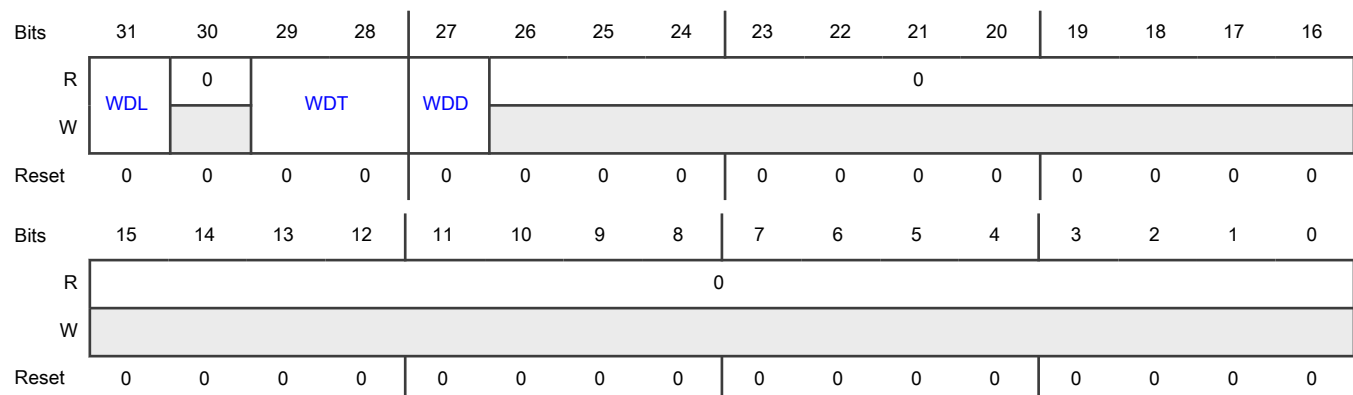
Offset

Register	Offset
WWD0G	3FCh

Function

Write watchdog timer control register. This is common register to control watchdog timer for both write buffers.

Diagram



Fields

Field	Function
31 WDL	Watchdog Lock Lock for write to all fields in this register. 0b - No lock. All fields in this register can be written without any restriction. 1b - Lock enabled. All fields in this register, including this field, can not be written until next reset.
30 —	Reserved
29-28 WDT	Watch Dog Tune Provides a mechanism to tune watchdog timer for write sequence for both write buffers. This field can not be written if WWDOG[WDL] is 1. 00b - Timeout set to 40000 register bus clocks. 01b - Timeout set to 200 register bus clocks. 10b - Timeout set to 400 register bus clocks. 11b - Timeout set to 4000 register bus clocks.
27 WDD	Watchdog Disable Provides a mechanism to disable watchdog timer for write sequence for both write buffers. This field can not be written if WDL field is 1 in this register. 0b - Watchdog timer enabled. 1b - Watchdog timer disabled.
26-0 —	Reserved

7.20 Write Data (WDATA0_PAGE0_WORD0 - WDATA1_PAGE7_WORD7)

Offset

For $l = 0$ to 1; $m = 0$ to 7; $n = 0$ to 7:

Register	Offset
WDATAI_PAGE m _WORD n	$400h + (l \times 200h) + (m \times 20h) + (n \times 4h)$

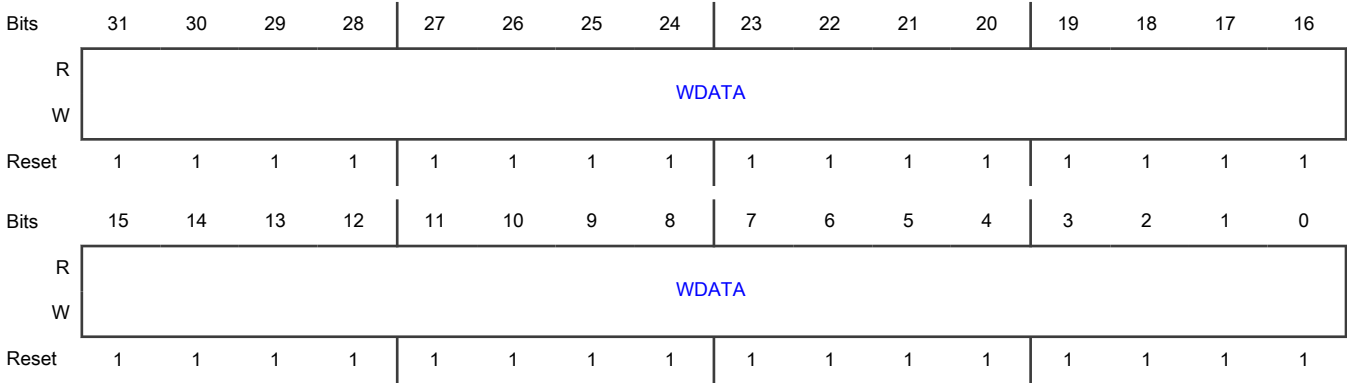
Function

Enables data to be written to MRAM. This set of 64 data registers (WDATAI_PAGE0-7_WORD0-7) enables upto 256 bytes of data to be written to MRAM at once. There are 2 sets of such registers WDATA0 and WDATA1.

WDATAI_PAGE0_WORD0 corresponds to lowest word address (4 bytes), and WDATAI_PAGE0_WORD0[7:0] corresponds to lowest byte address.

WDATAI_PAGE0_WORD0-7 are used for single page write.
WDATAI_PAGE0-3_WORD0-7 are used for 4 pages write.
WDATAI_PAGE0-7_WORD0-7 are used for 8 pages write.

Diagram



Fields

Field	Function
31-0 WDATA	Write Data Write data to be written to MRAM. Write access to this field is restricted to the domain ID that owns the write sequence indicated in WSTATn[WID].

7.21 Write Strobe (WSTRB0_PAGE0 -WSTRB1_PAGE7)

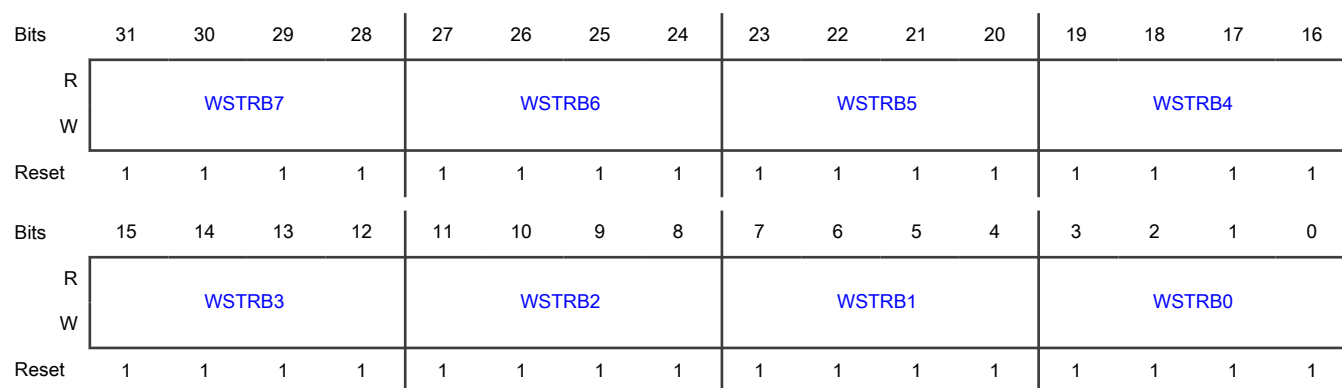
Offset

For l = 0 to 1; m = 0 to 7:

Register	Offset
WSTRBI_PAGEm	500h + (l × 200h) + (m × 4h)

Function

Controls byte strobe or enable for data to be written to MRAM. There is one write strobe page register corresponding to WDATAI_PAGEm_WORD7-0 data registers that covers one page (32 bytes). For example, WSTRB0_PAGE0 register corresponds to WDATA0_PAGE0_WORD7-0 registers.

Diagram**Fields**

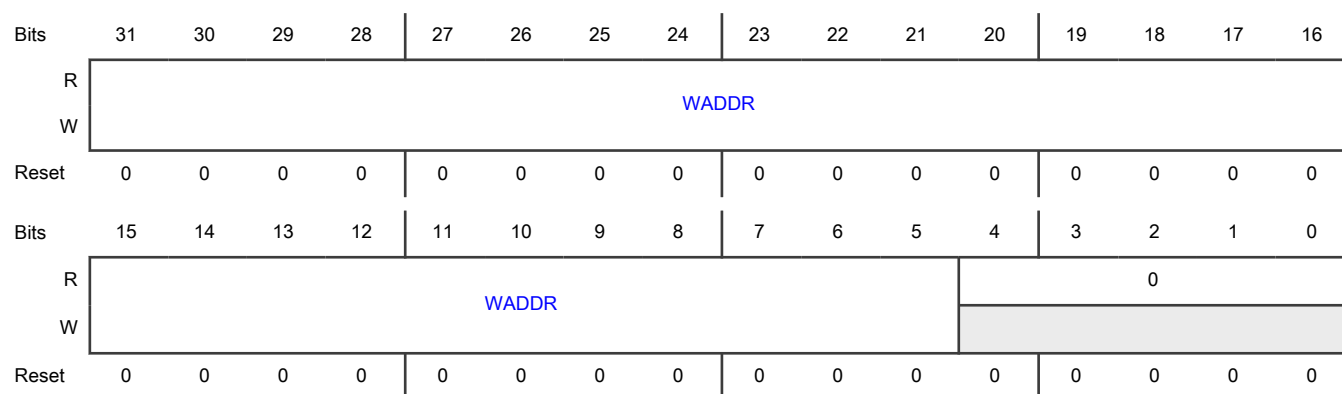
Field	Function
31-28: WSTRB7	Write Strobe
27-24: WSTRB6	Each bit value 1 indicates corresponding write data byte is valid.
23-20: WSTRB5	Each bit value 0 indicates corresponding write data byte is not valid.
19-16: WSTRB4	WSTRB0 field corresponds to WDATAI_PAGEm_WORD0 register, and bit WSTRB0[0] corresponds to byte WDATAI_PAGEm_WORD0[7:0].
15-12: WSTRB3	
11-8: WSTRB2	WSTRB1 field corresponds to WDATAI_PAGEm_WORD1 register, and bit WSTRB1[0] corresponds to byte WDATAI_PAGEm_WORD1[7:0], and so on.
7-4: WSTRB1	
3-0: WSTRB0	Write access to this field is restricted to the domain ID that owns the write sequence indicated in WSTATn[WID].

7.22 Write Address (WADDR0 - WADDR1)**Offset**

Register	Offset
WADDR0	520h
WADDR1	720h

Function

Provides the address to be written to MRAM. There is one write address register corresponding to set of 64 write data registers that covers upto 256 bytes. For example, WADDR0 corresponds to write data 0 page 0-7 registers. There are 2 sets of such write address registers.

Diagram**Fields**

Field	Function
31-5 WADDR	Write Address Provides address location to be written to MRAM. This is page address as visible to system. It should be set to start address of single or multi-page write chunk. For example, if 4 pages need to be written then address should be 4 pages aligned. Write sequence is locked to domain ID on successful write to this register. The owner of write sequence is updated in WSTATn[VID].
4-0 —	Reserved

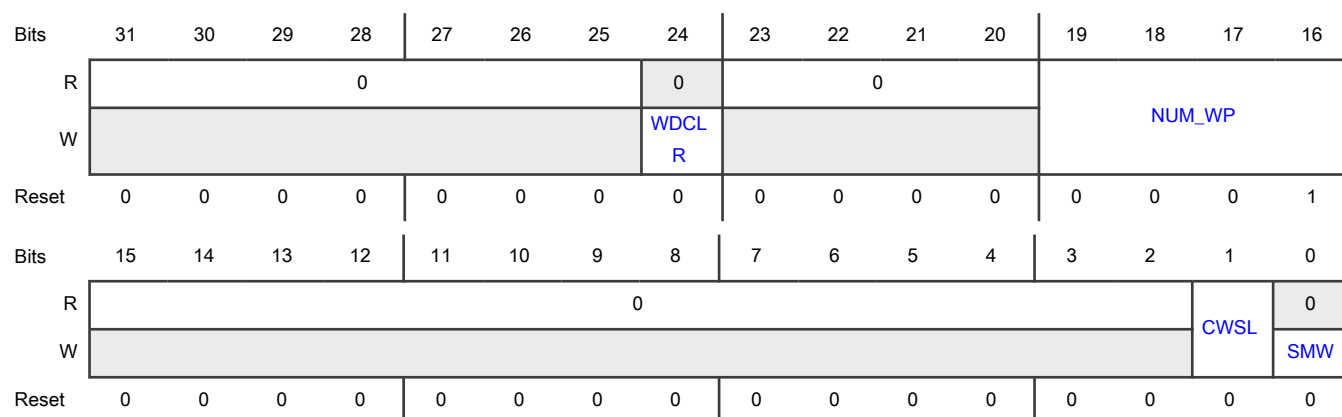
7.23 Write Control (WCTL0 - WCTL1)**Offset**

Register	Offset
WCTL0	524h
WCTL1	724h

Function

Provides a way to control write to MRAM from register interface. There is one write control register corresponding to set of 64 write data registers that covers upto 256 bytes. For example, WCTL0 corresponds to write data 0 page 0-7 registers. There are 2 sets of such write control registers.

Diagram



Fields

Field	Function
31-25 —	Reserved
24 WDCLR	Write Data Register Clear Clears all bits in all write data and write strobe page registers to value 1. This is write only field and reads 0. Write access to this field is restricted to the domain ID that owns the write sequence indicated in WSTATn[WID]. 0b - No effect. 1b - Clears write data page registers.
23-20 —	Reserved
19-16 NUM_WP	Number of Write Pages Defines number of pages to be written to MRAM. Write access to this field is restricted to the domain ID that owns the write sequence indicated in WSTATn[WID]. 0001b - Write single page. 0100b - Write four pages. 1000b - Write eight pages. All other values - Reserved.
15-2 —	Reserved
1 CWSL	Cancel Write Sequence Lock

Table continues on the next page...

Table continued from the previous page...

Field	Function
	Cancels the write sequence lock if the sequence is owned by a domain ID. Domain ID sequence restriction is lifted when this field is written to 1. This field need to be written before writing WCTL0[SMW], if you want to cancel write sequence lock. Writing 1 after SMW is already 1 does not have any effect on sequence lock. This field need to be set to 0, if is set to 1, before next write sequence is initiated. Write access to this field is restricted to the domain ID that owns the write sequence indicated in WSTATn[WID]. 0b - No effect. 1b - Cancels write sequence lock.
0 SMW	Start MRAM Write Starts MRAM write operation. All the write operation register fields must be valid before writing this field. Fields in this register can be written the same time as writing this field. This is a write only field which reads 0. Write access to this field is restricted to the domain ID that owns the write sequence indicated in WSTATn[WID]. Additionally, this field can not be written if write sequence lock has not been aquired. 0b - No effect. 1b - Start MRAM write operation.

7.24 Write Status (WSTAT0 - WSTAT1)

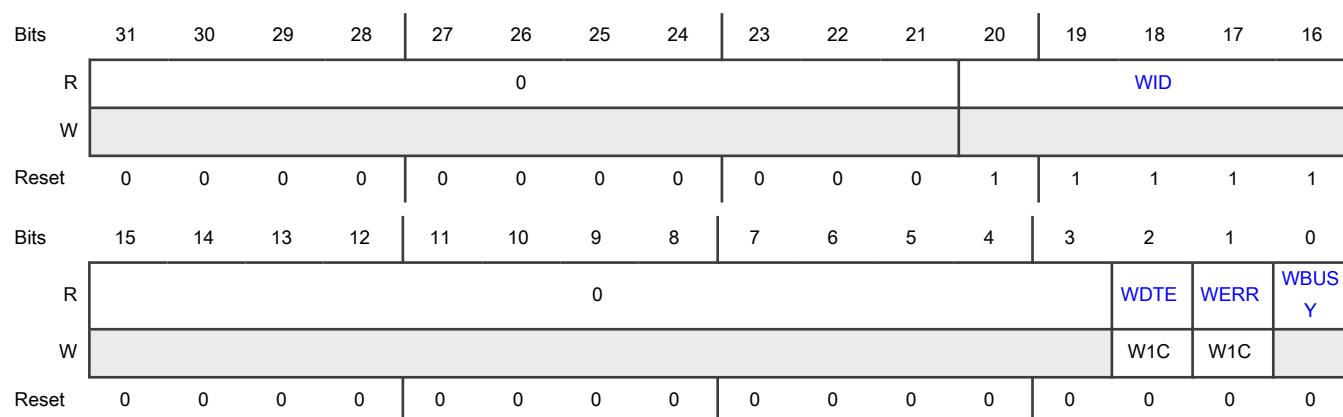
Offset

Register	Offset
WSTAT0	528h
WSTAT1	728h

Function

Provides a way to check status of write to MRAM from register interface. There is one write status register corresponding to set of 64 write data registers that covers upto 256 bytes. For example, WSTAT0 corresponds to write data 0 page 0-7 registers. There are 2 sets of such write status registers.

Diagram



Fields

Field	Function
31-21 —	Reserved
20-16 WID	Write ID This field shows the domain ID of the master that owns the write sequence. The ID is latched when the sequence has started (writing of the WADDR register). This field returns to default value (all 1s) on completion of write operation or write sequence lock release. 1_1111b - None of the domain ID is owning the write sequence. All other values - This domain id is owning the write sequence.
15-3 —	Reserved
2 WDTE	Watchdog Timeout Error Set by hardware to indicate that the write sequence is not completed by your software within time configured in watchdog timer and it timed out. The timeout releases write sequence lock restriction. Software need to clear it by writing value 1. 0b - No watchdog timeout. 1b - Watchdog timed out.
1 WERR	Write Error Set by hardware to indicate that the write to MRAM through register interface encountered error. Software needs to clear it by writing value 1. 0b - No error on MRAM write. 1b - Error encountered on write to MRAM.
0 WBUSY	Write Busy

Table continues on the next page...

Table continued from the previous page...

Field	Function
	Set by hardware to indicate that the write to MRAM through register interface is in progress. Cleared by hardware once write is finished. 0b - Write is not in progress. 1b - Write is in progress.

7.25 Debug Status 0 (DBGSTAT0)

Offset

Register	Offset
DBGSTAT0	FF0h

Function

Contains non-customer debug status information of MRAMC.

Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	Reserved															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved												STAT			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fields

Field	Function
31-3 —	Reserved
2-0 STAT	Status This field shows debug status information. [2:0]: Page write state machine state

7.26 Debug Status 1 (DBGSTAT1)

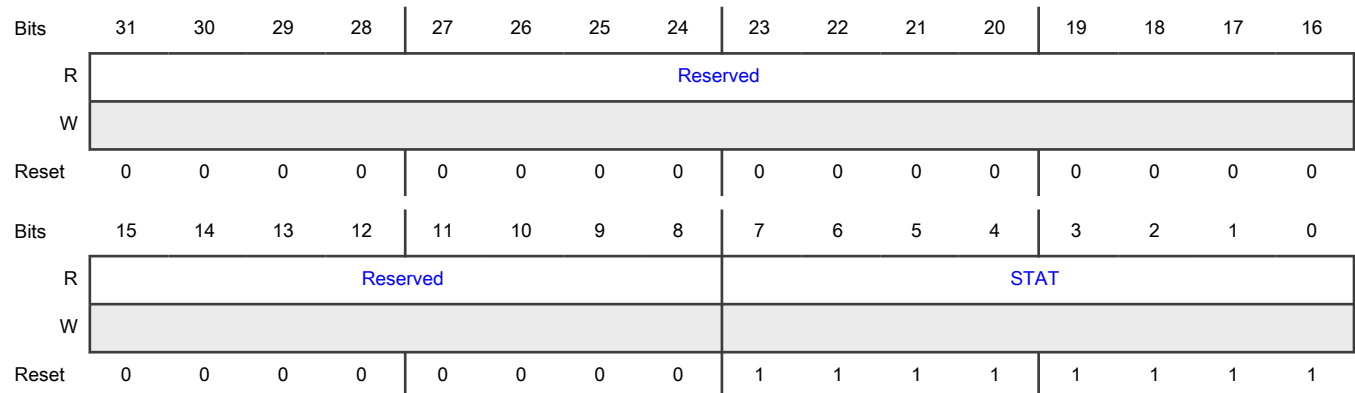
Offset

Register	Offset
DBGSTAT1	FF4h

Function

Contains non-customer debug status information of MRAMC.

Diagram



Fields

Field	Function
31-8 —	Reserved
7-0 STAT	Status Shows debug status information. [7:3]: AXI buffers empty [3:0]: AXI ports idle

7.27 Debug Status 2 (DBGSTAT2)

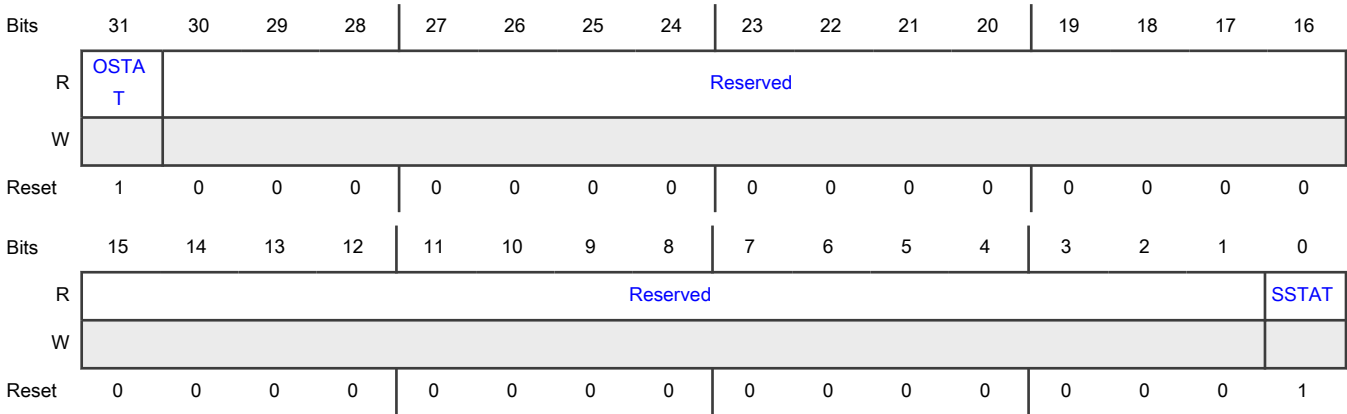
Offset

Register	Offset
DBGSTAT2	FF8h

Function

Contains non-customer debug status information of MRAMC.

Diagram



Fields

Field	Function
31 OSTAT	OCOTP status Shows debug status information. [0]: OCOTP idle
30-1 —	Reserved
0 SSTAT	Scheduler status Shows debug status information. [0]: Response static queue empty

7.28 Debug Status 3 (DBGSTAT3)

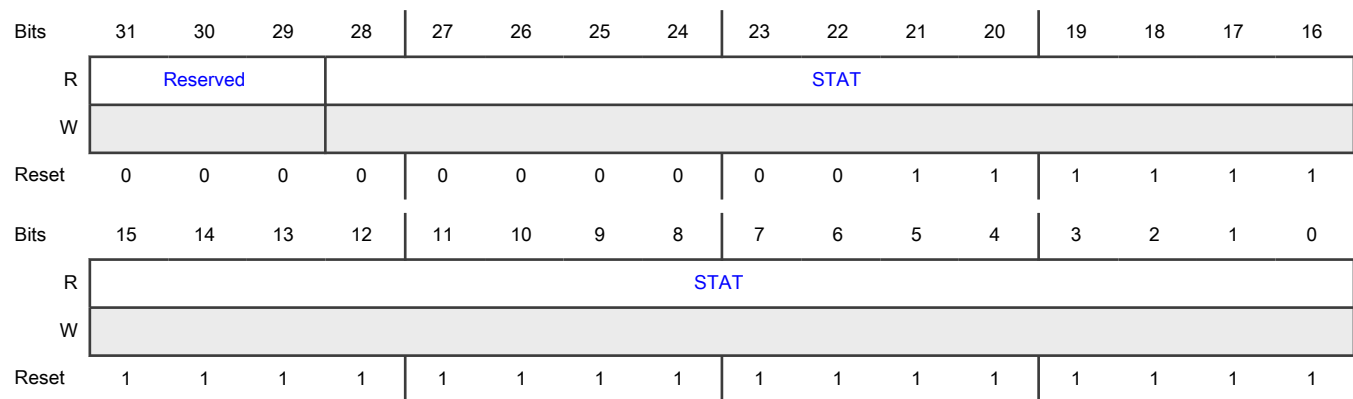
Offset

Register	Offset
DBGSTAT3	FFCh

Function

Contains non-customer debug status information of MRAMC.

Diagram



Fields

Field	Function
31-29 —	Reserved
28-0 STAT	Status Shows debug status information. [28] MRAM write enable pfl_wr_en [27:26] Number of writes sent to the FIFO mrm_wr_fifo_2: t1_multi_wr_num_ff[1:0] [25] Multi-page write FSM state t1_multi_fsm_ff [24:22] Write FSM state write_fsm_ff[2:0] [21] Write response FIFO mrm_wrsp_fifo empty [20] 3rd write FIFO mrm_wr_ong_fifo empty [19] 2nd write FIFO mrm_wr_fifo_2 empty [18] 1st write FIFO mrm_wr_fifo_1 empty [17:15] Read response FIFO mrm_rrsp_u_fifo empty[2:0] [14:12] Read response FIFO mrm_rrsp_l_fifo empty[2:0] [11:9] 2nd read FIFO mrm_rr_ong_u_fifo empty[2:0] [8:6] 2nd read FIFO mrm_rr_ong_l_fifo empty[2:0] [5:3] 1st read FIFO mrm_rr_u_fifo empty[2:0] [2:0] 1st read FIFO mrm_rr_l_fifo empty[2:0]

8 Glossary

AHB	Advanced High-performance Bus
ECC	Error Correcting Code
HSE	Hardware Security Engine
IPS	Internal Peripheral System

ID	Identification
MRAM	Magnetic Random Access Memory
OTA	Over the Air

9 MRAMC module changes (showing internal content)

Label	Date	Author	Changes (internal)	Changes (external)
	2022-09-29	Prakash Makwana	Initial BG with just 2 registers PC0-1.	
ATXSOF T.MRAM C_SPEC. REL_0_1 _INT	2022-09-29	Prakash Makwana		
	2022-10-04	Prakash Makwana	Added Lock registers.	
ATXSOF T.MRAM C_SPEC. REL_0_2 _INT	2022-10-04	Prakash Makwana		
	2023-06-01	Prakash Makwana	Updated according to D_IP_MRAMC_AXI_SYN IP RS - Revision MRAMC_1.1.	
ATXSOF T.MRAM C_SPEC. REL_1_0	2023-06-01	Prakash Makwana		
	2023-10-03	Prakash Makwana	- Added PC3 register - Changed number of large partitions from 10 to 16 affecting large partition registers. - Removed small partition OTP registers.	
ATXSOF T.MRAM C_SPEC. REL_1_2 _INT	2023-10-03	Prakash Makwana		
	2023-10-11	Prakash Makwana	Changed offset of LPOLOCKOW and UTLOCKOW registers due to RAL generation issue.	

Table continues on the next page...

Table continued from the previous page...

Label	Date	Author	Changes (internal)	Changes (external)
ATXSOF T.MRAM C_SPEC. REL_2_0	2023-10-11	Prakash Makwana		
	2023-10-13	Prakash Makwana	Updated functional description section.	
ATXSOF T.MRAM C_SPEC. REL_2_1	2023-10-13	Prakash Makwana		
	2023-11-20	Prakash Makwana	Added multi page write registers.	
ATXSOF T.MRAM C_SPEC. REL_2_2 _INT	2023-11-20	Prakash Makwana		
	2024-01-16	Rahul Goel	Revised the revision_history.dita to resolve the QA error.	
ATXSOF T.MRAM C_SPEC. REL_2_3	2024-01-16	Rahul Goel		
	2024-01-18	Rahul Goel	Resolved the QA errors.	
ATXSOF T.MRAM C_SPEC. REL_2_4	2024-01-18	Rahul Goel		
	2024-01-18	Rahul Goel	Updated the revision_history.dita.	
ATXSOF T.MRAM C_SPEC. REL_2_5	2024-01-18	Rahul Goel		
	2024-02-15	Prakash Makwana	Added Error injection and status registers.	
ATXSOF T.MRAM	2024-02-15	Prakash Makwana		

Table continues on the next page...

Table continued from the previous page...

Label	Date	Author	Changes (internal)	Changes (external)
C_SPEC. REL_2_6				
	2024-02-23	Prakash Makwana	Added Safety and register write sections.	
ATXSOF T.MRAM C_SPEC. REL_2_7	2024-02-23	Prakash Makwana		
	2024-03-01	Luciana Carvalho	Added the registers Pi_RIA_Wj for AXI Port i Read ID Assignment.	
ATXSOF T.MRAM C_SPEC. REL_2_8	2024-03-01	Luciana Carvalho		
	2024-03-05	Prakash Makwana	Updated IPS write buffer registers for sequence lock and XRDC.	
ATXSOF T.MRAM C_SPEC. REL_3_0	2024-03-05	Prakash Makwana		
	2024-03-20	Prakash Makwana	Added WWDOG register.	
ATXSOF T.MRAM C_SPEC. REL_3_1	2024-03-20	Prakash Makwana		
	2024-04-24	Rahul Goel	Updated the revision_history.dita from tagging "ATXSOF.T.MRAMC_SPEC.REL_2_5" and onwards.	
ATXSOF T.MRAM C_SPEC. REL_3_2	2024-04-24	Rahul Goel		
	2024-04-26	Prakash Makwana	- Added interrupt section. - Updated ERRSTAT0 register for interrupt or FCCU.	

Table continues on the next page...

Table continued from the previous page...

Label	Date	Author	Changes (internal)	Changes (external)
			Updated WSTAT WERR and WDTE fields.	
ATXSOF T.MRAM C_SPEC. REL_3_3	2024-04-26	Prakash Makwana		
	2024-05-03	Prakash Makwana	Separated row for tag in revision history.	
ATXSOF T.MRAM C_SPEC. REL_3_4	2024-05-03	Prakash Makwana		
	2024-05-13	Prakash Makwana	Added ERRNTCTL0 and DBGSTAT0-3 registers.	
ATXSOF T.MRAM C_SPEC. REL_4_0	2024-05-24	Prakash Makwana		
	2024-06-04	Prakash Makwana	Widened LOCKOWNER register field to DID_W+1, and changed reset value to all 1s.	
ATXSOF T.MRAM C_SPEC. REL_4_1 _INT	2024-06-04	Prakash Makwana		
	2024-07-03	Prakash Makwana	- Added details to memory organization section. - Removed MRAM IPS access steps from OTP commit.	
ATXSOF T.MRAM C_SPEC. REL_4_2 _INT	2024-07-03	Prakash Makwana		
	2024-07-19	Rahul Goel	Updated the block guide as per BGAC, Modnaming and Regnaming guidelines.	

Table continues on the next page...

Table continued from the previous page...

Label	Date	Author	Changes (internal)	Changes (external)
ATXSOF T.MRAM C_SPEC. REL_5_0	2024-07-19	Rahul Goel		
	2024-07-26	Prakash Makwana	- Renamed WCTL[COMMIT] register field to WCTL[SMW]. - Added clarification on PLCK field. - Added more details in Write buffer and ERRSTAT0 registers. - Updated DBGSTAT3 register. - Changed reset value of WCTL[NUM_WP] and LPOLOCK[LCK] registers.	

© NXP B.V. 2024.

For more information, please visit: <https://www.nxp.com>

All rights reserved.

Date of release: Aug-07-2024